

PROYECTO TECNOLÓGICO

Desarrollo en Sistemas de Información y Ciberseguridad

EILOR

Analizador táctico de espectro 2.4 GHz, radar BLE y portal de registro

Sistema basado en ESP32, Node.js y WebSocket en tiempo real

DESARROLLO DE PROYECTO TECNOLÓGICO EN SISTEMAS DE INFORMACIÓN Y
CIBERSEGURIDAD

Autor

Pablo Alejandro Tandazo Zarango

Tutor / docente

Ing. Marco Silva, Mg

Ciudad, provincia

23 de septiembre de 2026



**SISTEMAS
DE INFORMACIÓN
Y CIBERSEGURIDAD**

Resumen ejecutivo

El presente proyecto documenta el diseño, la implementación y la validación de **EILOR**, un sistema integral de *monitoreo del espectro radioeléctrico de 2.4 GHz* y *gestión de acceso de invitados* orientado a pequeños negocios y a laboratorios de aprendizaje en ciberseguridad. El sistema combina un dispositivo de hardware de bajo costo basado en el microcontrolador **ESP32**, que integra ambas radios y escanea redes Wi-Fi (canales 1 al 13) y balizas Bluetooth de Baja Energía (BLE) en modos dedicados conmutables, con un **servidor de tiempo real** construido sobre Node.js, Express y WebSocket que agrega, analiza, persiste y visualiza los datos capturados en un tablero de mando (*dashboard*) web.

EILOR incorpora cuatro modos operativos —análisis de espectro Wi-Fi, radar de proximidad BLE por RSSI, punto de acceso (SoftAP) con *portal cautivo* de registro de invitados y modo de telemetría/reposo— gobernados de forma remota mediante comandos WebSocket. En la capa de análisis, el servidor calcula un *puntaje de interferencia* por canal que pondera el solapamiento de canales adyacentes y la intensidad de señal, recomienda el canal más limpio del conjunto no solapado 1/6/11 y detecta puntos de acceso sospechosos de tipo *gemelo malicioso* (*evil twin*). Adicionalmente, se implementó una capa de seguridad con autenticación por token de sesión para las acciones de control, un token opcional de dispositivo para restringir la ingesta de datos y un mecanismo de identificación de fabricante mediante prefijos OUI, junto con la detección de direcciones MAC aleatorizadas por privacidad.

La validación funcional confirmó la operación de extremo a extremo del sistema: ingesta de escaneos, análisis de canales, persistencia con recuperación tras reinicio, control de acceso (respuestas 401/200 según credenciales) y registro de invitados desde el portal cautivo con *buffer* de reenvío ante caídas de enlace. El documento presenta, además, el análisis de desempeño, la estimación de costos con curva S, la evaluación de impacto ambiental, la propuesta de escalabilidad y los manuales de usuario y técnico, cumpliendo la totalidad de los entregables definidos en la macro actividad.

Palabras clave: espectro 2.4 GHz, ESP32, IEEE 802.11, Bluetooth Low Energy, WebSocket, portal cautivo, ciberseguridad, gemelo malicioso, OUI, tiempo real.

Abstract. This project documents the design, implementation and validation of **EILOR**, an integrated system for *2.4 GHz radio-frequency spectrum monitoring* and *guest-access management* aimed at small businesses and cybersecurity learning labs. It couples a low-cost **ESP32**-based device—which integrates both radios and scans Wi-Fi networks (channels 1–13) and Bluetooth Low Energy (BLE) beacons in dedicated switchable modes—with a **real-time server** built on Node.js, Express and WebSocket that aggregates, analyses, persists and visualises the captured data on a web dashboard. EILOR provides four operating modes, an adjacency-aware channel-interference score, evil-twin access-point detection, OUI-based vendor identification, randomized-MAC awareness, and a token-based security layer. End-to-end validation confirmed data ingestion, channel analysis, restart-safe persistence, access control and captive-portal guest registration with offline buffering.

Índice general

Resumen ejecutivo	ii
1. Introducción	1
1.1. Justificación	2
1.2. Alcance y delimitación	2
1.3. Estructura del documento	2
2. Planteamiento y delimitación del problema	3
2.1. Descripción del problema	3
2.2. Contextualización	3
2.2.1. Nivel macro	3
2.2.2. Nivel meso	4
2.2.3. Nivel micro	4
2.3. Formulación del problema	4
2.4. Objetivos preliminares y viabilidad	5
3. Marco teórico y antecedentes	6
3.1. El espectro de 2.4 GHz y la interferencia de canal	6
3.2. Wi-Fi (IEEE 802.11) y escaneo pasivo/activo	6
3.3. Bluetooth Low Energy (BLE)	7
3.4. Estimación de distancia por RSSI	7
3.5. Identificación de fabricante por OUI y MAC aleatorizadas	7
3.6. Seguridad inalámbrica: gemelo malicioso y buenas prácticas	8
3.7. Tecnologías de software de tiempo real	8
3.8. Síntesis de antecedentes	8
4. Objetivos	10
4.1. Objetivo general	10
4.2. Objetivos específicos	10
4.3. Matriz de objetivos bajo criterios SMART	11

5. Propuesta técnica y arquitectura	12
5.1. Visión general de la arquitectura	12
5.2. Descripción de las capas	13
5.2.1. Capa de dispositivo (ESP32)	13
5.2.2. Capa de servidor (Node.js)	13
5.2.3. Capa de clientes	13
5.3. Selección de tecnologías	13
5.4. Especificaciones técnicas iniciales	14
6. Diseño detallado	15
6.1. Diagrama de componentes	15
6.2. Flujo de datos: ciclo de escaneo	15
6.3. Flujo del portal cautivo con <i>buffer</i> offline	16
6.4. Modelo de datos	16
6.5. Especificación de interfaces	17
6.5.1. API REST	17
6.5.2. Mensajería WebSocket	17
7. Diseño electrónico y hardware	18
7.1. Arquitectura de hardware	18
7.2. Asignación de pines (pinout)	19
7.3. Lista de materiales (BOM)	19
7.4. Diseño conceptual de la carcasa	20
7.5. Consideraciones de alimentación y coexistencia RF	20
8. Diseño de la base de datos	22
8.1. Estrategia de persistencia	22
8.2. Modelo entidad-relación	22
8.3. Esquema físico (archivos JSON)	23
8.3.1. Ejemplo de documento (registro de invitado)	24
8.4. Ruta de migración a base de datos relacional	24
9. Implementación del prototipo	26
9.1. Programación del firmware (Arduino / ESP32)	26
9.1.1. Estructura de modos y bucle principal	26
9.1.2. Escaneo de espectro Wi-Fi	29
9.1.3. Portal cautivo y <i>buffer</i> offline	29
9.2. Implementación del servidor (Node.js)	31
9.2.1. Módulo de análisis de canales	31
9.2.2. Detección de gemelo malicioso	32

9.2.3. Identificación de fabricante (OUI)	32
9.2.4. Autenticación por token	33
9.2.5. Ingesta y difusión en tiempo real	33
9.3. Implementación del frontend (dashboard)	34
10.Despliegue SaaS en la nube con Cloudflare	35
10.1. Modelo de entrega como servicio (SaaS)	35
10.2. Arquitectura del túnel	35
10.3. Ventajas de seguridad	36
10.4. Configuración de referencia	36
10.5. Gestión del proceso con PM2	36
11.Plataforma web: interfaz, uso y comunicación	38
11.1. Acceso a la plataforma	38
11.1.1. Inicio de sesión	38
11.2. Estructura y opciones de la interfaz	39
11.2.1. Controlador de modos	39
11.2.2. Despliegue del punto de acceso (SoftAP)	40
11.2.3. Análisis del espectro y radar BLE	41
11.2.4. Telemetría y flujo de comandos	42
11.2.5. Inventario de dispositivos	42
11.2.6. Registros del portal cautivo	43
11.3. Diseño de la interfaz web	43
11.4. Comunicación entre la web, el servidor y el ESP32	44
11.5. Vista pública en producción	45
12.Pruebas de validación y seguridad	46
12.1. Estrategia de pruebas	46
12.2. Casos de prueba funcionales	46
12.3. Casos de prueba de seguridad	47
12.4. Análisis de amenazas y mitigaciones	47
12.5. Métricas de calidad y desempeño observadas	47
12.6. Evidencia de ejecución	48
13.Evaluación de desempeño	50
13.1. Metodología de evaluación	50
13.2. Cumplimiento de objetivos	50
13.3. Análisis de latencia y capacidad	50
13.4. Robustez y disponibilidad	51
13.5. Comparación con los objetivos planteados	51

14. Análisis económico y ambiental	53
14.1. Estimación de costos	53
14.2. Curva S del proyecto	53
14.3. Análisis de impacto ambiental	54
14.4. Relación costo-beneficio	54
15. Optimización y escalabilidad	55
15.1. Oportunidades de optimización identificadas	55
15.2. Plan de escalabilidad	55
15.2.1. Escalabilidad de datos	55
15.2.2. Escalabilidad de dispositivos	56
15.2.3. Escalabilidad de servicio	56
15.2.4. Endurecimiento de seguridad	56
15.3. Hoja de ruta	56
16. Manuales de usuario y técnico	57
16.1. Manual de usuario	57
16.1.1. Requisitos previos y preparación	57
16.1.2. Secuencia de arranque del ESP32	57
16.1.3. Acceso al dashboard	58
16.1.4. Inicio de sesión	58
16.1.5. Modos del dispositivo	58
16.1.6. Lectura de espectro y radar	59
16.1.7. Despliegue del punto de acceso y portal	59
16.1.8. Registros de invitados	60
16.1.9. Indicadores y telemetría	61
16.2. Manual técnico	62
16.2.1. Flujo técnico de extremo a extremo	62
16.2.2. Estructura del proyecto	64
16.2.3. Instalación y ejecución del servidor	64
16.2.4. Variables de entorno	64
16.2.5. Carga del firmware	64
16.2.6. Prueba técnica de aceptación	65
16.2.7. Publicación con Cloudflare Tunnel	65
16.2.8. Resolución de problemas frecuentes	65
16.2.9. Mantenimiento, respaldo y seguridad	66
17. Conclusiones y recomendaciones	67
17.1. Conclusiones generales y limitaciones	67
17.2. Recomendaciones	68

Referencias bibliográficas	70
A. Glosario de términos	72
B. Referencias del código fuente	73
C. Repositorio y despliegue de referencia	74

Índice de figuras

5.1. Arquitectura general de EILOR: dispositivo, túnel, servidor modular, persistencia y clientes. La imagen resume el flujo; los contratos de mensajes se especifican en las tablas y diagramas de secuencia del capítulo. . . .	12
6.1. Diagrama de componentes de software (servidor y firmware).	15
6.2. Diagrama de secuencia del ciclo de escaneo y control.	16
6.3. Flujo de registro del portal cautivo con reenvío ante caída de enlace. . .	16
7.1. Diagrama funcional de conexiones y alimentación autónoma de EILOR. Los colores identifican alimentación positiva (rojo), tierra (negro) y señales (amarillo); la posición gráfica de los terminales no sustituye el pinout de la Tabla 7.1.	18
7.2. Propuesta conceptual de carcasa para EILOR con compartimento de pilas, acceso a USB y zona libre frente a la antena.	20
7.3. Montaje físico del prototipo (ESP32 + OLED sobre <i>protoboard</i>) y monitor serie durante la puesta en marcha.	21
8.1. Modelo entidad-relación conceptual de EILOR.	23
9.1. Pantalla OLED en modo escaneo Wi-1-----Fi (modo 0): total de redes, canal más saturado y canal recomendado.	28
9.2. Pantalla OLED en modo escaneo BLE (modo 1): dispositivos detectados y RSSI del más cercano.	29
9.3. Pantalla OLED en modo SoftAP: SSID emitido, canal, IP del portal (192.168.4.1) y contadores de clientes y registros.	31

9.4. Dashboard web en operación: mapa de saturación espectral Wi9.3. IMPLEMENTACIÓN DEL FRONTEND (DASHBOARD)19.3. IMPLEMENTACIÓN DEL FRONTEND (DASHBOARD)-9.3. IMPLEMENTACIÓN DEL FRONTEND (DASHBOARD)9.3. IMPLEMENTACIÓN DEL FRONTEND (DASHBOARD)9.3. IMPLEMENTACIÓN DEL FRONTEND (DASHBOARD)9.3. IMPLEMENTACIÓN DEL FRONTEND (DASHBOARD)9.3. IMPLEMENTACIÓN DEL FRONTEND (DASHBOARD)-Fi (canales 1–13) y radar de proximidad BLE, alimentados en vivo por WebSocket.	34
10.1. Publicación del servidor mediante Cloudflare Tunnel.	35
11.1. Formulario de inicio de sesión de la plataforma (habilita los comandos del ESP32).	39
11.2. Cabecera de estado: enlace del ESP32, modo activo y contadores de dispositivos y paquetes.	39
11.3. Controlador de modos: conmutación remota de las cuatro funciones del ESP32.	40
11.4. Panel de despliegue del SoftAP: selección de SSID y canal del punto de acceso propio.	40
11.5. Formulario del portal cautivo tal como lo ve el invitado en su teléfono (vista previa).	41
11.6. Saturación espectral Wi-1-----Fi (canales 1–13) y radar de proximidad BLE en tiempo real.	41
11.7. Panel de telemetría: consola de tramas WebSocket y serie temporal de actividad.	42
11.8. Inventario de dispositivos con fabricante por OUI, marcado de gemelo malicioso y de MAC aleatorizada.	42
11.9. Registros del portal cautivo en el <i>dashboard</i> , con exportación restringida a la sesión.	43
11.10Flujo de comunicación web ↔ servidor ↔ ESP32 (el servidor concentra ambos sentidos).	44
11.11Plataforma EILOR publicada en <code>esp.orellanadigitalco.trade</code> a través del túnel de Cloudflare.	45
12.1. Salida de consola del banco de pruebas automatizado: 20/20 casos superados, con los códigos de control de acceso (401/200) y las latencias medidas.	48

12.2. Registros de invitados visibles en el <i>dashboard</i> en tiempo real, con exportación a CSV restringida a la sesión autenticada.	49
13.1. Nivel de cumplimiento por objetivo específico (100 % = meta).	52
14.1. Curva S: avance acumulado planificado vs. real.	54
15.1. Hoja de ruta de evolución del sistema.	56
16.1. Indicadores de operación: conexión del ESP32, modo activo y contadores globales.	58
16.2. Selector de los cuatro modos disponibles para el ESP32.	59
16.3. Uso del mapa de saturación Wi-1-----Fi y del radar BLE para observar el entorno de 2.4 GHz.	59
16.4. Configuración del SoftAP: SSID, canal y activación controlada desde el dashboard.	60
16.5. Portal cautivo que visualiza el invitado al conectarse a la red propia. .	60
16.6. Registros del portal cautivo y controles de exportación y limpieza. . . .	61
16.7. Panel de telemetría y estado de la conexión del dispositivo.	61
16.8. Vista completa del <i>dashboard</i> con todas sus secciones (cabecera, controlador de modos, SoftAP, espectro, radar, telemetría, inventario y registros).	62
16.9. Vista de referencia para validar el flujo completo después de una instalación técnica.	63

Índice de tablas

3.1. Aporte de los antecedentes a los componentes de EILOR	9
4.1. Matriz SMART de los objetivos específicos	11
5.1. Pila tecnológica y justificación	14
6.1. Entidades y atributos del modelo de datos	17
6.2. Endpoints de la API REST	17
7.1. Asignación de pines del ESP32	19
7.2. Lista de materiales y costo estimado	19
8.1. Archivos de persistencia	23
11.1. Composición del frontend de la plataforma	43
12.1. Casos de prueba funcionales y resultados	46
12.2. Casos de prueba de seguridad (control de acceso)	47
12.3. Amenazas y controles implementados	47
12.4. Métricas de desempeño observadas	48
13.1. Cumplimiento de objetivos frente a los indicadores SMART	50
13.2. Latencia de procesamiento y difusión (100 ingestas consecutivas, servidor local)	51
14.1. Estimación de costos del proyecto	53
15.1. Oportunidades de optimización	55
16.1. Variables de entorno del servidor	64
B.1. Archivos de código fuente del proyecto	73

Capítulo 1

Introducción

La banda de 2.4 GHz constituye hoy uno de los recursos radioeléctricos más congestionados del entorno cotidiano. En ella coexisten redes Wi-Fi bajo el estándar IEEE 802.11, dispositivos Bluetooth y Bluetooth de Baja Energía (BLE), teléfonos inalámbricos, hornos de microondas y una densidad creciente de dispositivos del Internet de las Cosas (IoT). Esta coexistencia, sumada a que solo tres de los trece canales de 2.4 GHz (1, 6 y 11) no se solapan entre sí, provoca degradaciones de rendimiento difíciles de diagnosticar sin instrumentación especializada (Bianchi, [2000](#); Gast, [2005](#)). Los analizadores de espectro comerciales, si bien precisos, resultan costosos y poco accesibles para pequeños negocios, instituciones educativas y laboratorios de aprendizaje.

Paralelamente, la seguridad de las redes inalámbricas se ha vuelto una preocupación central. Ataques como el punto de acceso *gemelo malicioso* (*evil twin*), en el que un atacante replica el nombre de una red legítima para interceptar tráfico, explotan precisamente la naturaleza abierta del medio y la confianza del usuario (Callegati et al., [2009](#); OWASP Foundation, [2021](#)). Detectar la presencia de estos puntos de acceso y comprender la ocupación del espectro son capacidades formativas esenciales para quien se inicia en la ciberseguridad.

El proyecto **EILOR** nace en la intersección de ambas necesidades: ofrecer una plataforma *asequible, didáctica y funcional* que permita, por un lado, *visualizar y analizar* el espectro de 2.4 GHz en tiempo real y, por otro, ejercer un *control de acceso* responsable mediante un punto de acceso propio con portal cautivo de registro de invitados —al estilo del Wi-Fi de una cafetería—. Todo ello se construye sobre hardware de bajo costo (ESP32) y una pila de software abierta (Node.js, Express, WebSocket).

1.1. Justificación

La elección del microcontrolador ESP32 responde a que integra, en un mismo encapsulado y por un costo reducido, radios Wi-Fi y Bluetooth/BLE, capacidad de procesamiento suficiente para escaneo activo y una pila TCP/IP completa (Espressif Systems, 2023). Frente a soluciones cerradas, EILOR privilegia la *transparencia* (código propio y modular), la *reproducibilidad* (persistencia en archivos JSON sin dependencias nativas) y el *valor pedagógico*: cada componente —desde el cálculo de interferencia por canal hasta la autenticación por token— es observable y auditable.

1.2. Alcance y delimitación

El sistema se circunscribe a la banda de 2.4 GHz y a los canales de *advertising* de BLE (37, 38 y 39). No abarca la banda de 5 GHz ni la decodificación de tramas de datos de terceros; el escaneo Wi-Fi se limita a la información de baliza (SSID, BSSID, canal, RSSI y tipo de cifrado) que la propia API del ESP32 expone. La funcionalidad de punto de acceso y portal cautivo está concebida para operar sobre una *red propia*, con un nombre de red identificable y un aviso explícito de registro, en cumplimiento de principios éticos y de protección de datos; su uso para suplantar redes ajenas queda expresamente fuera del alcance autorizado del proyecto.

1.3. Estructura del documento

El documento integra los cuatro entregables de la macro actividad. El Capítulo 2 delimita el problema en sus niveles macro, meso y micro; el Capítulo 3 desarrolla el marco teórico; el Capítulo 4 formula los objetivos bajo criterios SMART; y el Capítulo 5 presenta la arquitectura y las tecnologías (Entrega 1). El Capítulo 6 detalla el diseño; el Capítulo 9 documenta la implementación con el código real del sistema; y el Capítulo 12 expone el plan y los resultados de pruebas de seguridad (Entrega 2). Los Capítulos 13 a 16 abordan el desempeño, el análisis económico-ambiental con curva S, la escalabilidad y los manuales (Entrega 3). Finalmente, el Capítulo 17 presenta las conclusiones y la sustentación (Entrega 4), seguido de las referencias y los anexos.

Capítulo 2

Planteamiento y delimitación del problema

2.1. Descripción del problema

La saturación de la banda no licenciada de 2.4 GHz produce, en entornos reales, síntomas recurrentes: caídas de rendimiento, latencias variables, desconexiones intermitentes y una experiencia de usuario deficiente. La causa suele ser invisible para el usuario final —interferencia co-canal y de canal adyacente, exceso de puntos de acceso compitiendo por el mismo canal, o dispositivos BLE saturando las frecuencias de *advertising*— y su diagnóstico exige instrumentación que la mayoría de pequeños negocios no posee. A esta problemática técnica se suma una dimensión de seguridad: la facilidad con que puede desplegarse un punto de acceso que imite a una red legítima hace que los usuarios sean vulnerables a la captura de información si no existen mecanismos de concientización y control (Callegati et al., 2009; OWASP Foundation, 2021).

El problema que aborda EILOR se formula así: *no existe, para el pequeño negocio o el laboratorio educativo, una herramienta asequible que permita simultáneamente (i) diagnosticar la ocupación e interferencia del espectro de 2.4 GHz en tiempo real, (ii) identificar dispositivos y posibles amenazas inalámbricas, y (iii) gestionar de forma responsable el acceso de invitados a una red propia.*

2.2. Contextualización

2.2.1. Nivel macro

A escala global, la proliferación de dispositivos IoT —proyectada en decenas de miles de millones de unidades— presiona de manera sostenida las bandas no licenciadas

(Akyildiz et al., 2006). La gestión eficiente del espectro se ha convertido en una línea de investigación consolidada, desde la radio cognitiva hasta las técnicas de *spectrum sensing* (Yücek & Arslan, 2009). En este contexto macro, disponer de instrumentos de medición democratizados es un facilitador tanto del rendimiento de las redes como de la formación de profesionales.

2.2.2. Nivel meso

En el ámbito de las pequeñas y medianas organizaciones (cafeterías, comercios, oficinas, instituciones educativas), la conectividad Wi-Fi es un servicio esperado, pero rara vez se administra con criterio técnico. La selección del canal se deja habitualmente al valor por defecto del router, y la red de invitados —cuando existe— no distingue entre usuarios ni recopila información de contacto útil para el negocio. La ausencia de visibilidad sobre el espectro y sobre los dispositivos presentes limita tanto la optimización del servicio como la respuesta ante incidentes.

2.2.3. Nivel micro

En el nivel micro —el de un local concreto o un laboratorio de aula— el problema se materializa en decisiones cotidianas: ¿en qué canal debería operar el punto de acceso para minimizar interferencia?, ¿cuántos y qué tipo de dispositivos hay alrededor?, ¿existe algún punto de acceso que suplante el nombre de la red del negocio?, ¿cómo registrar de forma sencilla a los invitados que se conectan? EILOR responde a estas preguntas concretas con una interfaz única y datos en vivo.

2.3. Formulación del problema

Pregunta general: ¿Cómo diseñar e implementar un sistema de bajo costo que permita el monitoreo en tiempo real del espectro de 2.4 GHz, la identificación de dispositivos y amenazas inalámbricas, y la gestión responsable del acceso de invitados mediante un portal cautivo?

Preguntas específicas:

- ¿Qué arquitectura de hardware y software satisface el escaneo de Wi-Fi y BLE (en modos dedicados conmutables) con visualización en tiempo real?
- ¿Qué algoritmo de análisis permite estimar la interferencia por canal y recomendar el canal óptimo?
- ¿Cómo detectar puntos de acceso de tipo gemelo malicioso e identificar el fabricante de un dispositivo a partir de su dirección MAC?

- ¿Qué mecanismos de seguridad y de protección de datos deben incorporarse para un portal de registro de invitados?

2.4. Objetivos preliminares y viabilidad

El problema es abordable con recursos accesibles: un ESP32 (costo aproximado inferior a USD 10), una pantalla OLED, un pulsador y un servidor Node.js ejecutable en cualquier equipo o en la nube. La viabilidad técnica se sustenta en la capacidad dual-radio del ESP32 (Espressif Systems, [2023](#)) y en la madurez de las tecnologías web de tiempo real (Fette & Melnikov, [2011](#)). Los objetivos formales se detallan en el Capítulo 4.

Capítulo 3

Marco teórico y antecedentes

Este capítulo sintetiza los fundamentos conceptuales y los antecedentes que sustentan el proyecto, organizados en cinco ejes: el medio radioeléctrico de 2.4 GHz, las tecnologías Wi-Fi y BLE, los modelos de análisis de espectro y de localización por RSSI, la seguridad inalámbrica y las tecnologías de software de tiempo real.

3.1. El espectro de 2.4 GHz y la interferencia de canal

La banda ISM de 2.4 GHz se divide, en la mayoría de las regiones, en trece canales Wi-Fi separados 5 MHz entre sí. Dado que cada canal ocupa aproximadamente 20–22 MHz, canales contiguos se solapan y solo el conjunto {1, 6, 11} resulta mutuamente no solapado (Gast, 2005). La interferencia se clasifica en *co-canal* (dos emisores en el mismo canal, que comparten el medio mediante CSMA/CA) y de *canal adyacente* (emisores en canales cercanos cuyo espectro se traslapa, degradando la relación señal-ruido). El modelo analítico de Bianchi (Bianchi, 2000) sobre la función de coordinación distribuida (DCF) de IEEE 802.11 explica por qué el rendimiento agregado decae al aumentar el número de estaciones que contienen por un canal, justificando la importancia de seleccionar canales poco congestionados.

3.2. Wi-Fi (IEEE 802.11) y escaneo pasivo/activo

El estándar IEEE 802.11 (IEEE, 2021) define los mecanismos de descubrimiento de redes: en el escaneo *pasivo*, la estación escucha las tramas *beacon* que los puntos de acceso emiten periódicamente; en el *activo*, envía *probe requests* y espera *probe responses*. De estas tramas se extraen el identificador de red (SSID), la dirección física del punto

de acceso (BSSID), el canal, la intensidad de señal recibida (RSSI) y el tipo de cifrado. EILOR utiliza el escaneo que ofrece la API del ESP32 para obtener estos campos sin decodificar tráfico de datos de terceros, respetando así la delimitación ética del proyecto.

3.3. Bluetooth Low Energy (BLE)

BLE, definido a partir de la especificación Bluetooth Core (Bluetooth SIG, 2023) y descrito en profundidad por Heydon (Heydon, 2013), emplea 40 canales de 2 MHz, de los cuales tres —los canales 37, 38 y 39, ubicados en 2402, 2426 y 2480 MHz— se reservan para *advertising* (anuncio de presencia). Un dispositivo BLE emite paquetes de *advertising* que otros pueden detectar sin establecer conexión, exponiendo un nombre, una dirección y una potencia de señal. EILOR aprovecha estos paquetes para su "radar de proximidad", estimando la cercanía relativa de las balizas a partir del RSSI.

3.4. Estimación de distancia por RSSI

La conversión de RSSI a distancia se apoya en el modelo de pérdida de trayectoria logarítmica (Rappaport, 2002), según el cual la potencia recibida decae proporcionalmente al logaritmo de la distancia:

$$RSSI(d) = RSSI(d_0) - 10 n \log_{10} \left(\frac{d}{d_0} \right), \quad (3.1)$$

donde n es el exponente de pérdida de trayectoria (típicamente entre 2 en espacio libre y 4 en interiores con obstáculos) y d_0 una distancia de referencia. Despejando d se obtiene una estimación de distancia que, aunque sensible al entorno, resulta suficiente para un posicionamiento radial cualitativo, tal como demostró el sistema RADAR de Bahl y Padmanabhan (Bahl & Padmanabhan, 2000), pionero en la localización en interiores basada en RSSI.

3.5. Identificación de fabricante por OUI y MAC aleatorizadas

Los primeros tres octetos de una dirección MAC constituyen el *Organizationally Unique Identifier* (OUI), asignado por la IEEE a cada fabricante (IEEE Registration Authority, 2017). Consultando el OUI puede inferirse el fabricante de un dispositivo. No obstante, desde hace varios años los sistemas operativos móviles emplean *aleatorización de MAC* para preservar la privacidad, generando direcciones localmente administradas —identificables porque el segundo bit menos significativo del primer octeto está a 1—

que impiden el rastreo persistente (Martin et al., 2017). EILOR implementa tanto la búsqueda de fabricante por OUI como la detección de direcciones aleatorizadas, informando al analista cuándo un dispositivo oculta su identidad real.

3.6. Seguridad inalámbrica: gemelo malicioso y buenas prácticas

El ataque de *gemelo malicioso* consiste en desplegar un punto de acceso que anuncia el mismo SSID que una red legítima para inducir a las víctimas a conectarse a él, habilitando ataques de intermediario (*man-in-the-middle*) (Callegati et al., 2009). Una heurística de detección consiste en identificar un mismo SSID anunciado por múltiples BSSID distintos, señal de posible suplantación. Los controles recomendados por marcos como OWASP (OWASP Foundation, 2021) —autenticación robusta, minimización de datos y transparencia hacia el usuario— guían el diseño de la capa de seguridad y del portal de registro de EILOR.

3.7. Tecnologías de software de tiempo real

La comunicación bidireccional de baja latencia entre el hardware, el servidor y los clientes web se sustenta en el protocolo WebSocket (Fette & Melnikov, 2011), que mantiene una conexión persistente sobre TCP evitando el costo de las peticiones HTTP repetidas. El servidor se organiza siguiendo el estilo arquitectónico REST (Fielding & Taylor, 2002) para sus endpoints de consulta y control, y se ejecuta sobre Node.js (OpenJS Foundation, 2024), cuyo modelo de E/S no bloqueante resulta idóneo para manejar numerosas conexiones concurrentes. El portal cautivo, por su parte, se apoya en la manipulación de las consultas DNS (Mockapetris, 1987) para redirigir a los clientes a la página de registro. Los fundamentos generales de redes que enmarcan estas tecnologías se encuentran en obras de referencia como las de Tanenbaum y Wetherall (Tanenbaum & Wetherall, 2011) y Kurose y Ross (Kurose & Ross, 2017).

3.8. Síntesis de antecedentes

La Tabla 3.1 resume cómo los antecedentes revisados aportan a cada componente de EILOR.

Tabla 3.1: Aporte de los antecedentes a los componentes de EILOR

Componente	Fundamento / antecedente	Referencia
Análisis de canal	Interferencia y DCF en 802.11	Gast (2005); Bianchi (2000)
Escaneo Wi-Fi	Tramas <i>beacon/probe</i>	IEEE (2021)
Radar BLE	Canales de <i>advertising</i>	Bluetooth SIG (2023); Heydon (2013)
Distancia por RSSI	Modelo de pérdida de trayectoria	Rappaport (2002); Bahl & Padmanabhan (2000)
Fabricante / MAC	OUI y aleatorización	IEEE Registration Authority (2017); Martin et al. (2017)
Detección <i>evil twin</i>	MITM y buenas prácticas	Callegati et al. (2009); OWASP Foundation (2021)
Comunicación en vivo	WebSocket y REST	Fette & Melnikov (2011); Fielding & Taylor (2002)
Portal cautivo	Redirección DNS	Mockapetris (1987)
Plataforma de servidor	Node.js, E/S no bloqueante	OpenJS Foundation (2024)

Capítulo 4

Objetivos

4.1. Objetivo general

Diseñar, implementar y validar un sistema de bajo costo, denominado EILOR, para el monitoreo en tiempo real del espectro de 2.4 GHz, la identificación de dispositivos y amenazas inalámbricas, y la gestión responsable del acceso de invitados mediante un punto de acceso con portal cautivo de registro, integrando hardware ESP32 y un servidor web de tiempo real.

4.2. Objetivos específicos

- OE1.** Desarrollar un firmware para ESP32 que escanee redes Wi-Fi (canales 1–13) y balizas BLE (canales 37–39) en modos dedicados conmutables y transmita los resultados en tiempo real mediante WebSocket.
- OE2.** Construir un servidor modular en Node.js que agregue los datos, calcule un puntaje de interferencia por canal, recomiende el canal óptimo y detecte puntos de acceso de tipo gemelo malicioso.
- OE3.** Implementar la identificación de fabricante por OUI y la detección de direcciones MAC aleatorizadas para enriquecer el inventario de dispositivos.
- OE4.** Incorporar una capa de seguridad con autenticación por token de sesión para el control y un token opcional de dispositivo para la ingesta de datos.
- OE5.** Desarrollar un punto de acceso (SoftAP) con portal cautivo que registre invitados (nombre, apellido y teléfono) y persista la información en el servidor, con reenvío ante caídas de enlace.

OE6. Validar funcionalmente el sistema completo mediante pruebas de ingesta, análisis, persistencia, control de acceso y registro de invitados.

4.3. Matriz de objetivos bajo criterios SMART

Cada objetivo específico se formuló siguiendo los criterios SMART (específico, medible, alcanzable, relevante y temporal). La Tabla 4.1 presenta la matriz de verificación.

Tabla 4.1: Matriz SMART de los objetivos específicos

OE	Indicador medible	Criterio de logro (alcanzable/relevante)	Plazo
OE1	N.º de canales escaneados y latencia de ciclo	13 canales Wi-Fi + 3 BLE; ciclo ≤ 3 s	Sem. 6
OE2	Puntaje por canal y canal recomendado	Cálculo para 13 canales y salida 1/6/11	Sem. 7
OE3	Cobertura del catálogo OUI	≥ 120 prefijos; detección de MAC aleatoria	Sem. 7
OE4	Respuestas de control autenticado	401 sin token / 200 con token	Sem. 7
OE5	Registros entregados por el portal	100 % de envíos con enlace; <i>buffer</i> sin enlace	Sem. 7
OE6	Casos de prueba superados	≥ 90 % de casos funcionales OK	Sem. 8

La justificación de cada objetivo se articula con las preguntas específicas del Capítulo 2: OE1 y OE2 responden a la necesidad de diagnóstico del espectro; OE3 y OE6 a la identificación de dispositivos y su validación; OE4 y OE5 a la gestión responsable y segura del acceso de invitados.

Capítulo 5

Propuesta técnica y arquitectura

5.1. Visión general de la arquitectura

EILOR adopta una arquitectura de tres capas físicas —*dispositivo*, *servidor* y *clientes*— comunicadas mediante WebSocket sobre TLS (WSS) a través de un túnel público, y complementadas por una API REST para consultas y control. La Figura 5.1 sintetiza el flujo.

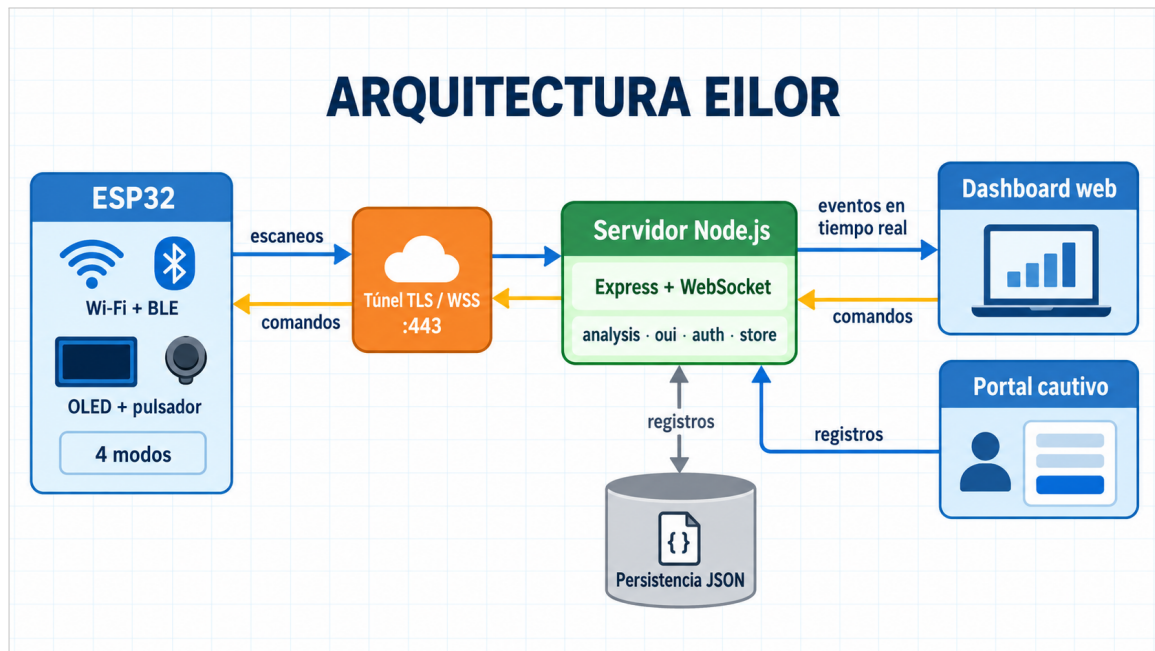


Figura 5.1: Arquitectura general de EILOR: dispositivo, túnel, servidor modular, persistencia y clientes. La imagen resume el flujo; los contratos de mensajes se especifican en las tablas y diagramas de secuencia del capítulo.

5.2. Descripción de las capas

5.2.1. Capa de dispositivo (ESP32)

El firmware implementa cuatro modos operativos conmutables por un pulsador físico o por comando remoto: (0) análisis de espectro Wi-Fi, (1) radar de proximidad BLE, (2) punto de acceso SoftAP con portal cautivo y (3) telemetría/reposo. Una pantalla OLED SSD1306 muestra el estado local, y el dispositivo mantiene una conexión WebSocket persistente con el servidor para transmitir escaneos y recibir comandos.

5.2.2. Capa de servidor (Node.js)

El servidor centraliza la lógica de negocio en cuatro módulos desacoplados dentro de `lib/`: `analysis` (puntuaje de interferencia y detección de gemelos maliciosos), `oui` (identificación de fabricante y MAC aleatorizadas), `store` (persistencia y series temporales) y `auth` (autenticación por token). Sobre ellos, `server.js` expone un servidor WebSocket y una API REST de catorce *endpoints*.

5.2.3. Capa de clientes

Dos tipos de clientes consumen el sistema: el *dashboard* web, que recibe actualizaciones en vivo por WebSocket y visualiza el espectro, el radar, la tabla de dispositivos y los registros; y los *invitados*, que al conectarse al SoftAP son redirigidos por el portal cautivo a un formulario de registro.

5.3. Selección de tecnologías

La Tabla 5.1 justifica la pila tecnológica seleccionada.

Tabla 5.1: Pila tecnológica y justificación

Capa	Tecnología	Justificación
Hardware	ESP32	Radios Wi-Fi + BLE integradas, bajo costo, pila TCP/IP (Espressif Systems, 2023)
Pantalla	OLED SSD1306	Bajo consumo, I ² C, suficiente para telemetría local
Firmware	C++ / Arduino	Ecosistema maduro de librerías (WebSockets, BLE, DNS-Server)
Servidor	Node.js + Express	E/S no bloqueante idónea para tiempo real (OpenJS Foundation, 2024)
Tiempo real	WebSocket (ws)	Canal bidireccional persistente de baja latencia (Fette & Melnikov, 2011)
Persistencia	Archivos JSON	Sin dependencias nativas; portable y auditable
Portal cautivo	DNSServer + WebServer	Redirección DNS y formulario embebido (Mockapetris, 1987)
Frontend	HTML/CSS/JS	Sin framework; ligero y transparente
Despliegue	PM2	Gestión de procesos, reinicio y persistencia de arranque

5.4. Especificaciones técnicas iniciales

- **Banda:** 2.4 GHz; canales Wi-Fi 1–13; canales BLE de advertising 37, 38 y 39.
- **Frecuencia de escaneo:** ciclo de ~ 3 s por modo activo.
- **Telemetría:** *heartbeat* cada 3 s; *watchdog* de desconexión a los 15 s.
- **Series temporales:** una muestra por minuto, retención de 720 muestras (~ 12 h).
- **Persistencia:** *snapshot* de dispositivos cada 30 s y volcado en apagado ordenado.
- **Seguridad:** token de sesión (TTL 24 h) para control; token de dispositivo opcional para ingesta.

Capítulo 6

Diseño detallado

6.1. Diagrama de componentes

La Figura 6.1 descompone el sistema en sus componentes de software y sus dependencias. En el servidor, `server.js` orquesta cuatro módulos especializados; en el firmware, el bucle principal coordina el escaneo, el portal cautivo y el enlace WebSocket.

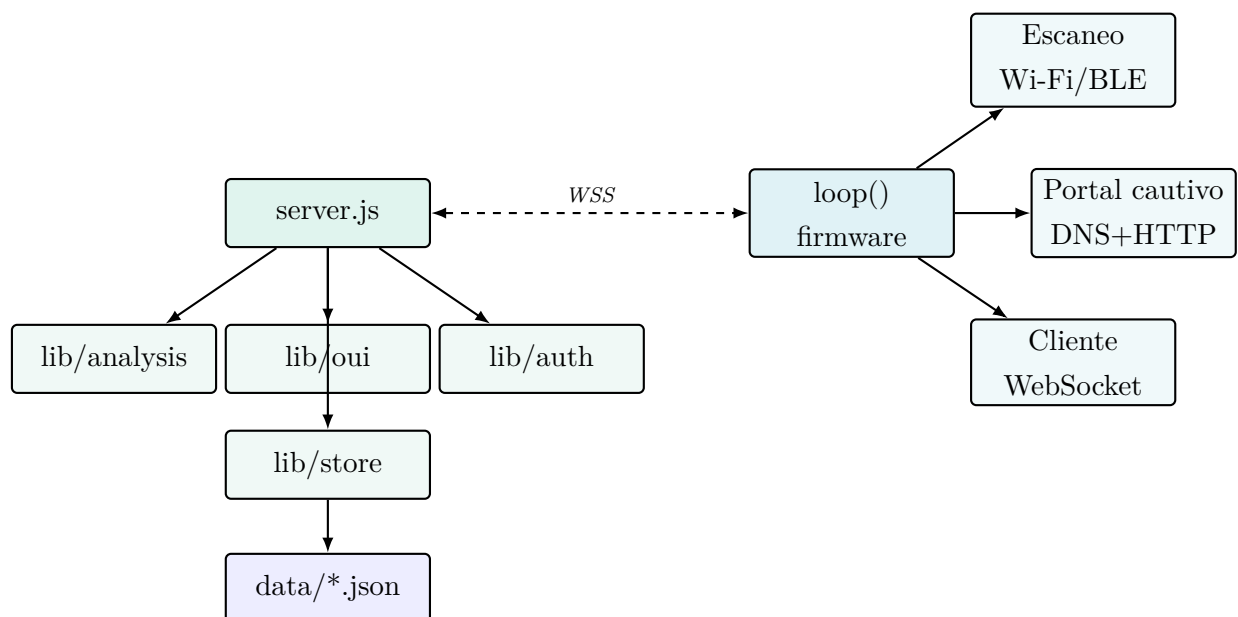


Figura 6.1: Diagrama de componentes de software (servidor y firmware).

6.2. Flujo de datos: ciclo de escaneo

La Figura 6.2 muestra la secuencia de un ciclo de escaneo Wi-Fi, desde la captura en el ESP32 hasta la actualización del dashboard.

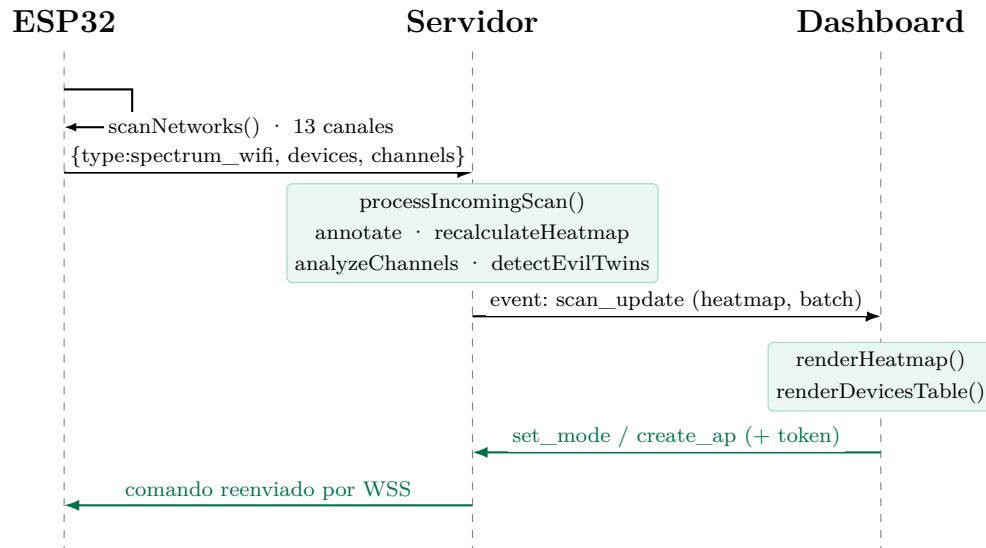


Figura 6.2: Diagrama de secuencia del ciclo de escaneo y control.

6.3. Flujo del portal cautivo con *buffer* offline

El registro de invitados sigue el flujo de la Figura 6.3. Si en el momento del envío no hay enlace con el servidor, el registro se almacena en una cola local y se reenvía automáticamente al reconectar, garantizando que no se pierdan altas.

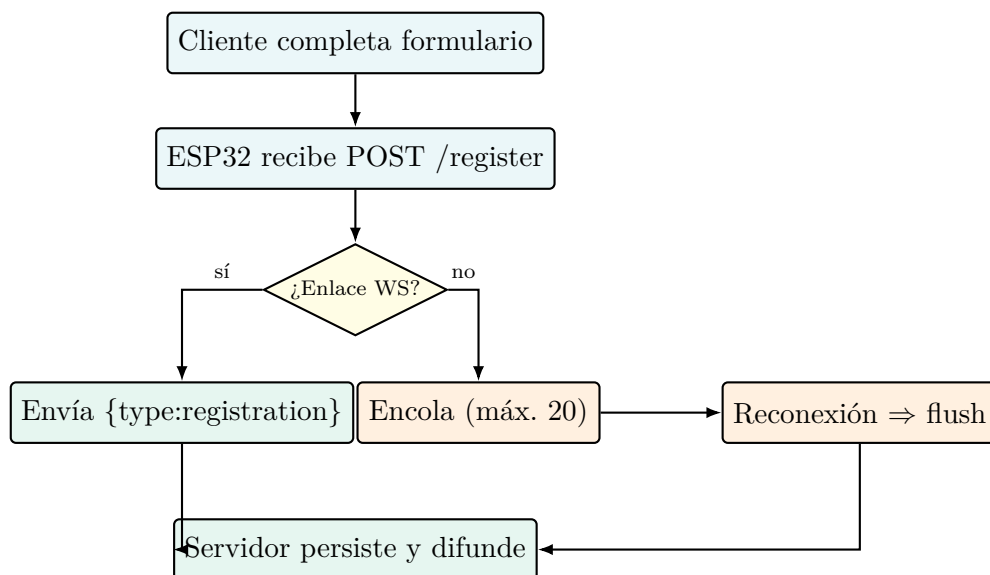


Figura 6.3: Flujo de registro del portal cautivo con reenvío ante caída de enlace.

6.4. Modelo de datos

El sistema maneja cuatro entidades principales, resumidas en la Tabla 6.1.

Tabla 6.1: Entidades y atributos del modelo de datos

Entidad	Atributos principales
Dispositivo	mac, type (wifi/ble), ssid, rssi, channel, encryption, distanceEst, vendor, randomMac, evilTwin, lastSeen
Mapa de calor	wifi[1..13]: {freq, count, maxRssi, avgRssi, ssids}; ble[37..39]: {freq, count, maxRssi}
Muestra temporal	t, wifi, ble, active, packets, rate
Registro de invitado	id, nombre, apellido, telefono, apSsid, source, ts

6.5. Especificación de interfaces

6.5.1. API REST

La Tabla 6.2 documenta los catorce *endpoints*, indicando el control de acceso aplicado (sesión de usuario, token de dispositivo o público).

Tabla 6.2: Endpoints de la API REST

Método	Ruta	Función	Acceso
POST	/api/login	Iniciar sesión, emite token	Público
POST	/api/logout	Cerrar sesión	Sesión
GET	/api/session	Validar token	Público
POST	/api/scan	Ingesta de escaneos	Dispositivo
GET	/api/scan	Estado y dispositivos	Público
POST	/api/mode	Cambiar modo del ESP32	Sesión
POST	/api/ap	Desplegar SoftAP	Sesión
GET	/api/heatmap	Mapa de calor de canales	Público
GET	/api/timeseries	Serie temporal	Público
POST	/api/registration	Alta de invitado	Dispositivo
GET	/api/registrations	Listar registros	Sesión
GET	/api/registrations.csv	Exportar CSV	Sesión
POST	/api/registrations/clear	Borrar registros	Sesión
POST	/api/clear	Reiniciar datos	Sesión

6.5.2. Mensajería WebSocket

El canal WebSocket transporta mensajes de cliente a servidor —`get_state`, `register/heartbeat`, `escaneos` (`type` wifi/ble), `set_mode/toggle_mode`, `create_ap`, `registration` y `ping`— y eventos de servidor a cliente —`initial_state`, `scan_update`, `esp32_status_change`, `telemetry_log`, `timeseries_sample`, `registration`, `registrations_cleared`, `clear`, `auth_error` y `pong`—. Las acciones de control validan el token de sesión y la ingesta valida el token de dispositivo antes de ejecutarse.

Capítulo 7

Diseño electrónico y hardware

7.1. Arquitectura de hardware

El hardware de EILOR se compone de tres elementos activos —la placa ESP32, una pantalla OLED y un pulsador— más la alimentación. El ESP32 actúa como unidad central: integra las radios de 2.4 GHz, gobierna la pantalla por el bus I²C y lee el pulsador por una entrada digital con resistencia *pull-up* interna. La Figura 7.1 presenta el diagrama de conexiones.

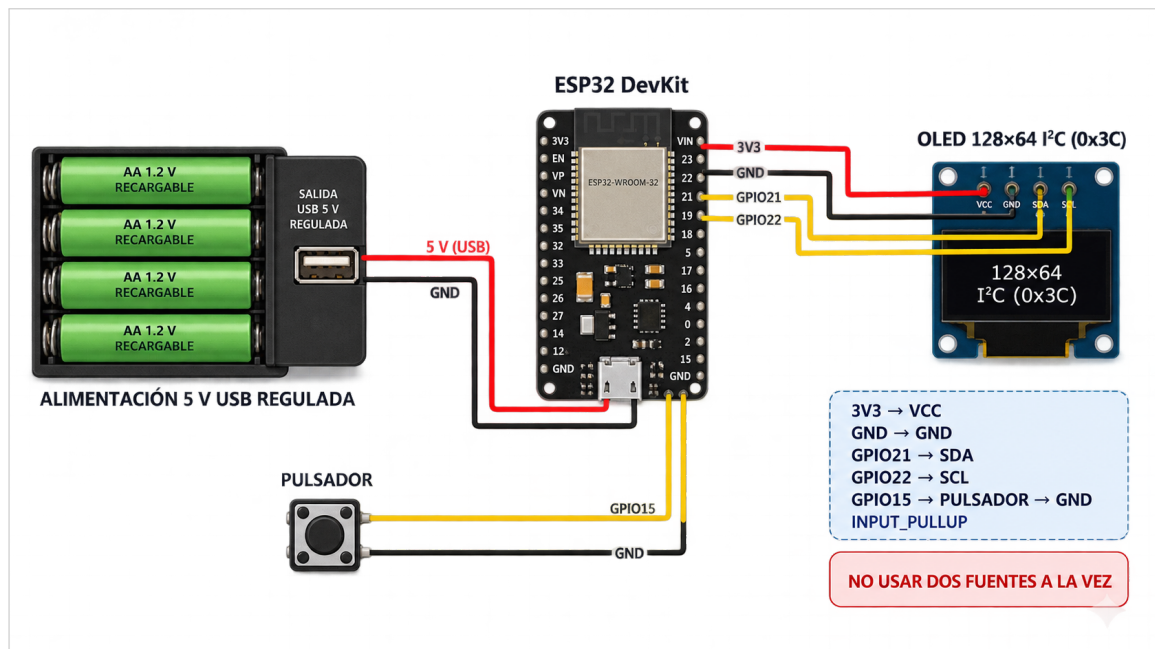


Figura 7.1: Diagrama funcional de conexiones y alimentación autónoma de EILOR. Los colores identifican alimentación positiva (rojo), tierra (negro) y señales (amarillo); la posición gráfica de los terminales no sustituye el pinout de la Tabla 7.1.

La figura fue generada con asistencia de inteligencia artificial y posteriormente contrastada con el firmware del proyecto y la documentación del fabricante. La verificación confirmó GPIO21 para SDA, GPIO22 para SCL y GPIO15 para el pulsador; asimismo, la guía de la placa establece que USB, 5V/GND y 3V3/GND son alternativas de alimentación mutuamente excluyentes (Espressif Systems, 2026). Por ello, debe utilizarse una sola fuente: en operación autónoma, un portapilas con salida USB regulada a 5 V; durante programación, el USB del computador, desconectando previamente el portapilas.

7.2. Asignación de pines (pinout)

La Tabla 7.1 detalla las conexiones. El pulsador se conecta entre GPIO15 y GND; al configurarse la entrada como INPUT_PULLUP, el pin se lee en alto en reposo y en bajo al pulsar, eliminando la necesidad de una resistencia externa.

Tabla 7.1: Asignación de pines del ESP32

Pin ESP32	Señal	Componente	Notas
3V3	Alimentación	OLED VCC	Lógica y alimentación a 3.3 V
GND	Común	OLED / Pulsador	Referencia de tierra compartida
GPIO21	SDA (I ² C)	OLED	Datos del bus I ² C
GPIO22	SCL (I ² C)	OLED	Reloj del bus I ² C
GPIO15	Entrada digital	Pulsador	INPUT_PULLUP; activo en bajo

7.3. Lista de materiales (BOM)

La Tabla 7.2 presenta la lista de materiales con un costo estimado referencial.

Tabla 7.2: Lista de materiales y costo estimado

Componente	Cant.	Costo unit. (USD)	Subtotal (USD)
Placa ESP32 DevKit (WROOM-32)	1	6.50	6.50
Pantalla OLED SSD1306 0.96" I ² C	1	3.00	3.00
Pulsador momentáneo	1	0.20	0.20
Protoboard y cables Dupont	1	3.00	3.00
Cable USB de alimentación	1	1.50	1.50
Portapilas 4×AA con salida USB regulada a 5 V y pilas NiMH	1	10.00	10.00
Total estimado			24.20

7.4. Diseño conceptual de la carcasa

La Figura 7.2 propone una carcasa de dos piezas imprimible en 3D. El diseño separa el portapilas de la electrónica, mantiene accesibles la pantalla, el pulsador y el puerto USB, incorpora ventilación y reserva una zona sin metal frente a la antena impresa del ESP32. Los postes de montaje y canales internos inmovilizan las placas y reducen esfuerzos sobre los conductores.

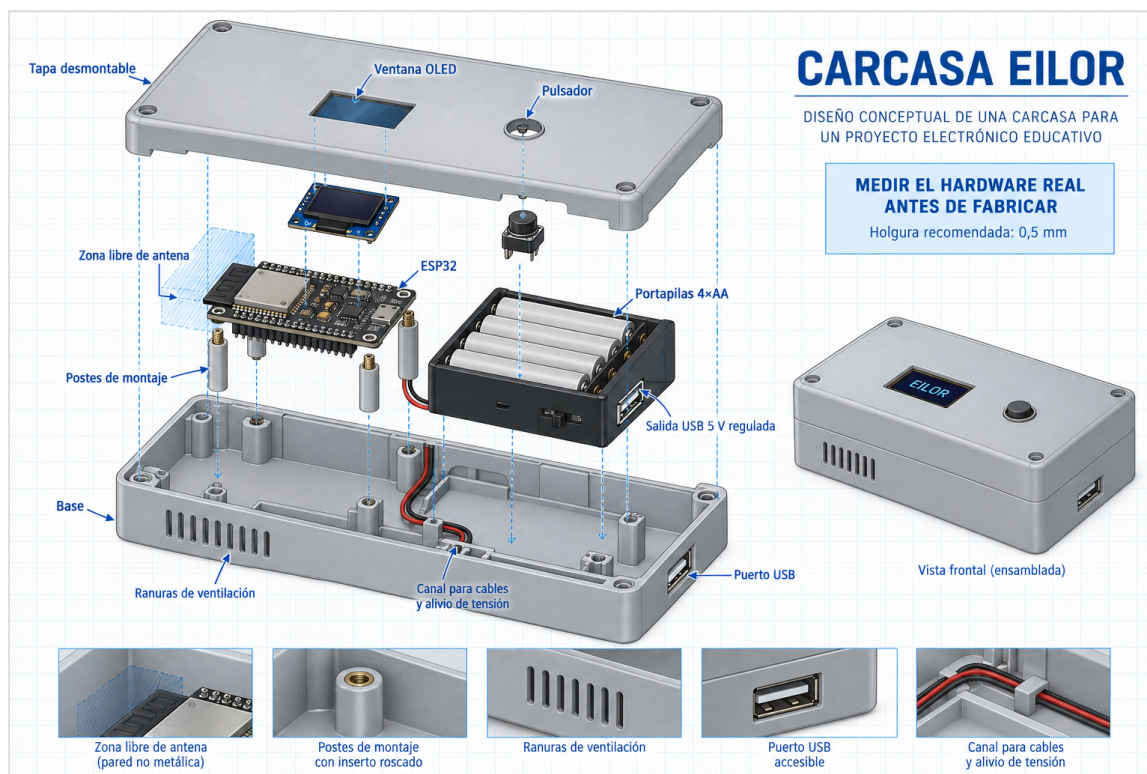


Figura 7.2: Propuesta conceptual de carcasa para EILOR con compartimento de pilas, acceso a USB y zona libre frente a la antena.

La imagen de carcasa es una referencia de distribución, no un plano listo para fabricación. Antes de modelar el archivo STL deben medirse con calibrador la placa, la OLED, el pulsador, el portapilas y la posición exacta de conectores y orificios. Se recomienda una holgura inicial de 0.5 mm por lado, prototipar primero la base y confirmar que ningún poste, tornillo o cable invada la zona de la antena.

7.5. Consideraciones de alimentación y coexistencia RF

El consumo del ESP32 durante el escaneo activo con ambas radios puede presentar picos de corriente; por ello se recomienda alimentación regulada de 5 V por USB capaz

de entregar al menos 500 mA. Las pilas AA no se conectan directamente a 3V3 ni a 5V: el portapilas debe integrar una salida USB regulada. En el firmware se habilita `WiFi.setSleep(true)` para favorecer la *coexistencia* temporal entre Wi-Fi y BLE, que comparten la misma radio de 2.4 GHz, evitando la inanición de una tecnología por la otra. La captura fotográfica del montaje físico se incorpora en el Anexo correspondiente.

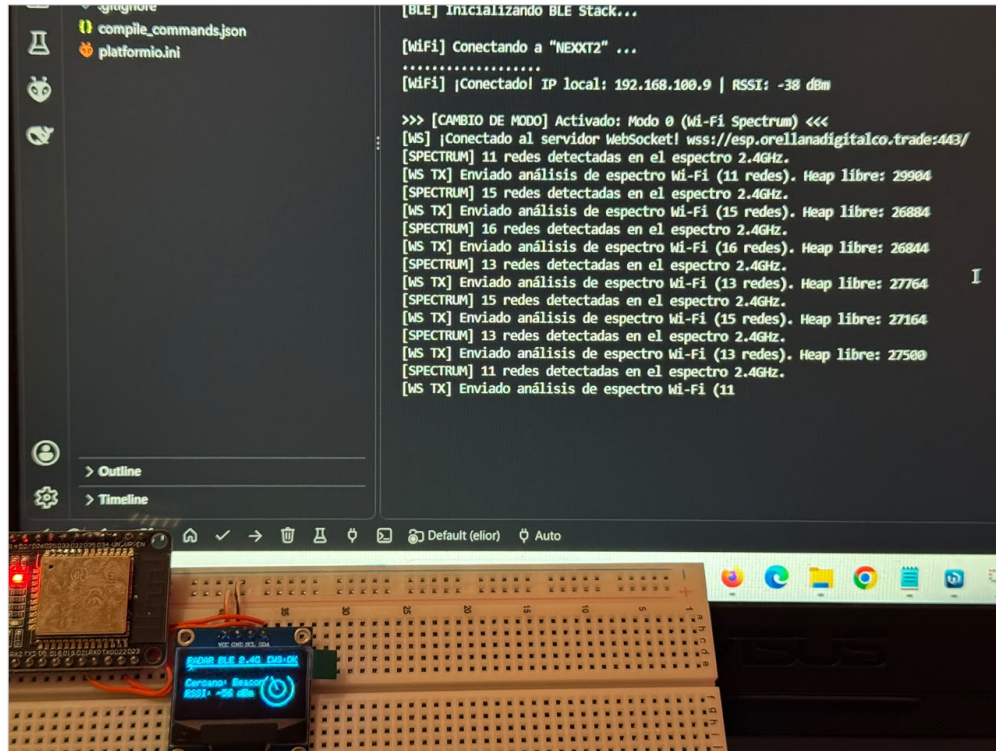


Figura 7.3: Montaje físico del prototipo (ESP32 + OLED sobre *protoboard*) y monitor serie durante la puesta en marcha.

Capítulo 8

Diseño de la base de datos

8.1. Estrategia de persistencia

Por decisión de diseño —priorizando la portabilidad, la ausencia de dependencias nativas y la facilidad de despliegue en cualquier entorno, incluida la nube— EILOR implementa su capa de datos mediante *persistencia en archivos JSON* gestionada por el módulo `lib/store`. Esta aproximación, adecuada para el volumen y la naturaleza del prototipo, se comporta como una base de datos documental ligera: cada colección se materializa en un archivo, con escritura atómica mediante archivo temporal y renombrado. La Sección 8.4 describe la ruta de migración a un motor relacional para escenarios de mayor escala.

8.2. Modelo entidad-relación

La Figura 8.1 presenta el modelo conceptual. Las entidades **Dispositivo**, **Muestra-Temporal** y **Registro** son independientes en el modelo documental actual, pero se relacionan lógicamente con la entidad **PuntoAcceso** (el SoftAP activo) y con la sesión de captura.

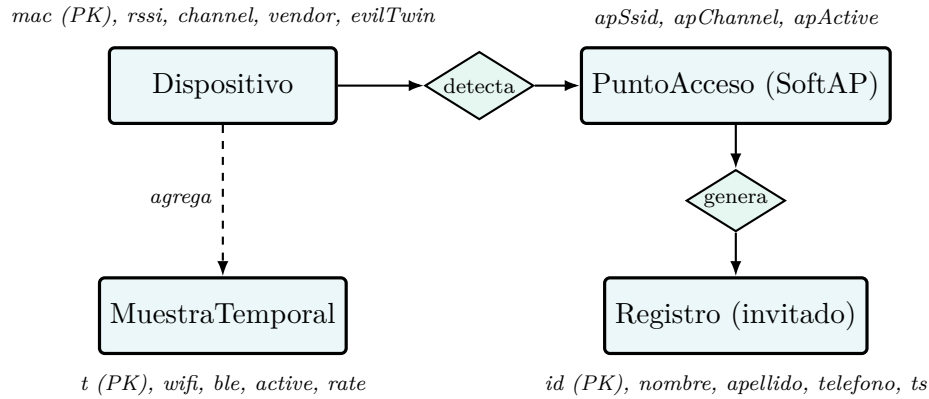


Figura 8.1: Modelo entidad-relación conceptual de EILOR.

8.3. Esquema físico (archivos JSON)

El módulo `lib/store` define tres archivos bajo el directorio `data/`, descritos en la Tabla 8.1.

Tabla 8.1: Archivos de persistencia

Archivo	Estructura	Contenido / política
<code>state.json</code>	Objeto	Instantánea de dispositivos y total de paquetes; <i>snapshot</i> cada 30 s
<code>timeseries.json</code>	Arreglo	Muestras por minuto; retención máx. 720 (MAX_TIMESERIES)
<code>registrations.json</code>	Arreglo	Altas de invitados; retención máx. 5000 (MAX_REGISTRATIONS)

El Listado 8.1 muestra la escritura atómica y el modelo de retención de la serie temporal.

```

1 function saveJSON(file, data) {
2   try {
3     ensureDir();
4     const tmp = file + '.tmp'; // escritura atomica:
5     fs.writeFileSync(tmp, JSON.stringify(data)); // 1) escribe
      temporal
6     fs.renameSync(tmp, file); // 2) renombra (
      atomico)
7     return true;
8   } catch (e) { return false; }
9 }
10
11 function saveTimeseries(series) {
12   return saveJSON(HISTORY_FILE, series.slice(-MAX_TIMESERIES)); //
      retencion

```

13 }

Listing 8.1: Escritura atómica y retención (lib/store.js)

8.3.1. Ejemplo de documento (registro de invitado)

```

1 {
2   "id": "mudcvuhsnl34",
3   "nombre": "Maria",
4   "apellido": "Perez",
5   "telefono": "099 123 4567",
6   "apSsid": "Cafe-EILOR",
7   "source": "esp32",
8   "ts": 1790121584629
9 }
```

Listing 8.2: Documento de la colección registrations.json

8.4. Ruta de migración a base de datos relacional

Para escenarios multi-dispositivo o de retención prolongada se propone migrar a un motor relacional (SQLite en despliegues embebidos o PostgreSQL en la nube). El Listado 8.3 presenta el esquema relacional equivalente, que preserva las entidades del modelo ER y agrega integridad referencial e índices para consultas eficientes.

```

1 CREATE TABLE dispositivo (
2   mac          VARCHAR(32) PRIMARY KEY,
3   tipo         VARCHAR(8)  NOT NULL,    -- 'wifi' / 'ble'
4   ssid         VARCHAR(64),
5   rssi         INTEGER,
6   channel      INTEGER,
7   vendor       VARCHAR(64),
8   random_mac   BOOLEAN DEFAULT FALSE,
9   evil_twin    BOOLEAN DEFAULT FALSE,
10  last_seen     BIGINT
11 );
12
13 CREATE TABLE registro (
14   id           VARCHAR(24) PRIMARY KEY,
15   nombre       VARCHAR(60) NOT NULL,
16   apellido     VARCHAR(60),
17   telefono     VARCHAR(30),
18   ap_ssid      VARCHAR(40),
19   fuente       VARCHAR(16),
20   ts           BIGINT NOT NULL
21 );
```

```
22 CREATE INDEX idx_registro_ts ON registro (ts);
23
24 CREATE TABLE muestra_temporal (
25     t          BIGINT PRIMARY KEY,
26     wifi       INTEGER, ble INTEGER,
27     activos    INTEGER, paquetes BIGINT, tasa INTEGER
28 );
```

Listing 8.3: Esquema relacional propuesto

Esta migración es transparente para el resto del sistema gracias al desacoplamiento que ofrece `lib/store`: bastaría reimplementar sus funciones (`loadDevices`, `saveDevices`, `loadTimeseries`, `saveTimeseries`, `loadRegistrations`, `saveRegistrations`) sobre el nuevo motor, sin alterar `server.js`.

Capítulo 9

Implementación del prototipo

Este capítulo documenta la implementación real del sistema. El proyecto suma más de 3 900 líneas de código distribuidas entre el firmware del ESP32 (777 líneas), el servidor Node.js (665 líneas más 274 en los módulos `lib/`) y el frontend (913 líneas de lógica, 419 de estructura y 1 272 de estilos). Se presentan los fragmentos más representativos; los archivos completos se referencian en los anexos y se acompañan de capturas donde procede.

9.1. Programación del firmware (Arduino / ESP32)

El firmware se desarrolló en C++ sobre el *core* de Arduino para ESP32, empleando las librerías `WiFi`, `WebServer`, `DNSServer`, `WebSocketsClient` (Markus Sattler), la pila BLE nativa y `Adafruit_SSD1306` para la pantalla. El bucle principal coordina el escaneo, el *heartbeat*, la lectura del pulsador y, en modo SoftAP, la atención del portal cautivo.

9.1.1. Estructura de modos y bucle principal

El Listado 9.1 muestra el despacho por modo dentro de `loop()`.

```
1 void loop() {
2     websocket.loop();
3     button.loop();
4
5     // Heartbeat cada 3 s hacia el servidor
6     if (isWsConnected && (millis() - lastHeartbeatTime > 3000)) {
7         websocket.sendTXT("{\"action\":\"heartbeat\" + tokenField() +
8             ...");
9         lastHeartbeatTime = millis();
10    }
```

```
10  if (button.isPressed()) applyMode((current_mode + 1) % 4); //
    pulsador D15
11
12  switch (current_mode) {
13      case 0: if (millis() - lastScanTime > 3000) {
                scanAndAnalyzeSpectrum(); lastScanTime = millis(); } break;
14      case 1: if (millis() - lastScanTime > 3000) { scanAndStreamBLE();
                lastScanTime = millis(); } break;
15      case 2: if (captiveRunning) { dnsServer.processNextRequest();
                captiveServer.handleClient(); } break;
16      case 3: break; // standby
17  }
18 }
```

Listing 9.1: Bucle principal y despacho por modo (firmware)

El cambio de modo se realiza físicamente con el pulsador (GPIO15); en cada pulsación el firmware avanza al siguiente modo y refresca la pantalla OLED con la información propia de ese estado. Las Figuras siguientes muestran el dispositivo físico operando en los modos de escaneo Wi-Fi y BLE.

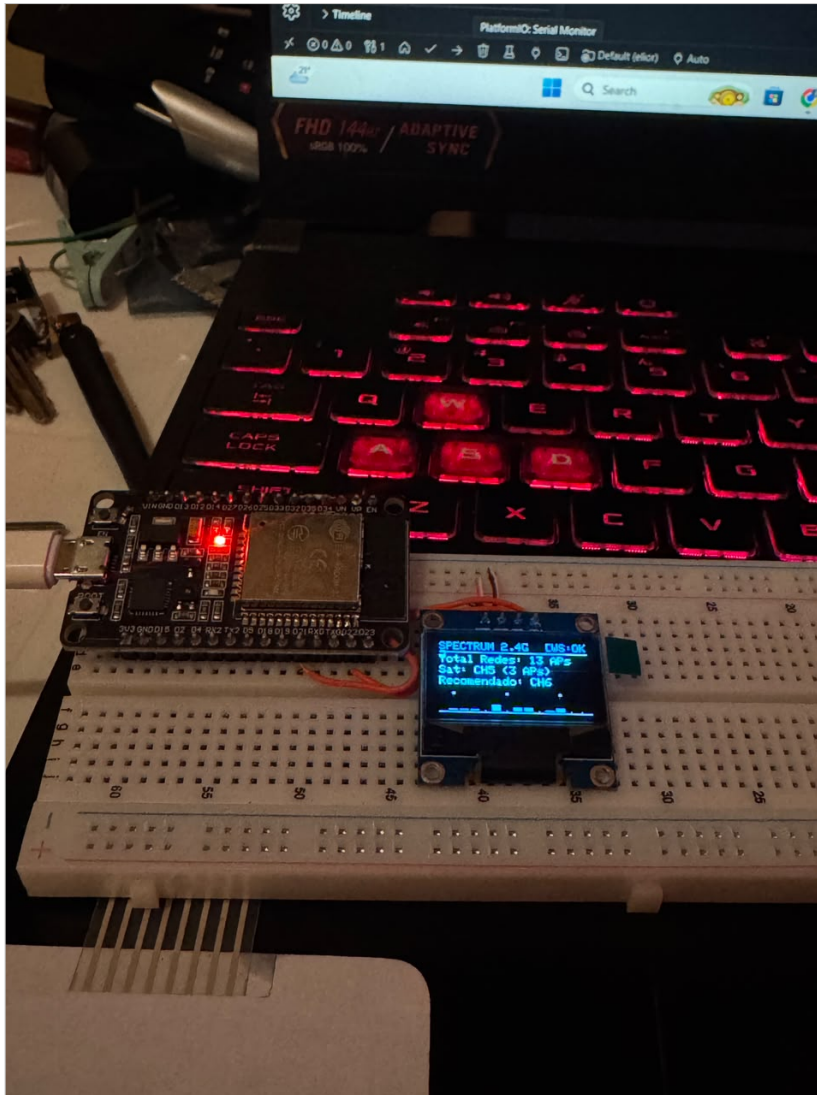


Figura 9.1: Pantalla OLED en modo escaneo Wi-Fi (modo 0): total de redes, canal más saturado y canal recomendado.

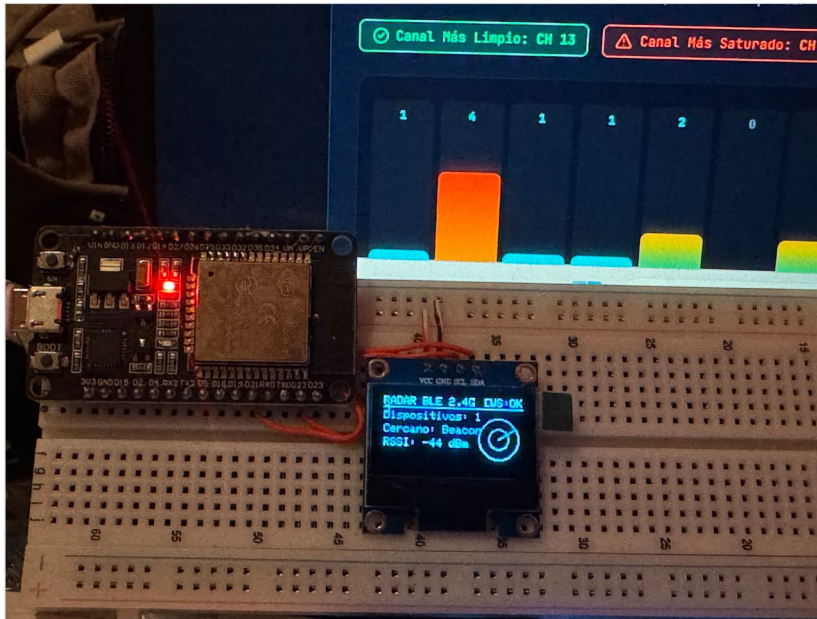


Figura 9.2: Pantalla OLED en modo escaneo BLE (modo 1): dispositivos detectados y RSSI del más cercano.

9.1.2. Escaneo de espectro Wi-Fi

En el modo 0, el firmware realiza un escaneo asíncrono, cuenta los puntos de acceso por canal y construye una trama JSON con los dispositivos y el histograma de canales, que envía por WebSocket (Listado 9.2).

```

1 String json = "{\"type\":\"spectrum_wifi\" + tokenField() +
2             \",\"deviceId\":\"ESP32-EILOR\", \"devices\":[";
3 for (int i = 0; i < n; ++i) {
4     int ch = WiFi.channel(i);
5     if (ch >= 1 && ch <= 13) channelCounts[ch]++;
6     if (i > 0) json += ",";
7     json += "{\"ssid\":\"" + WiFi.SSID(i) + "\",\"
8             \"bssid\":\"" + WiFi.BSSIDstr(i) + "\",\"
9             \"rssi\":\"" + String(WiFi.RSSI(i)) + "\",\"
10            \"channel\":\"" + String(ch) + "\"}";
11 }
12 json += "],\"channels\":{\" ... } }";
13 if (isWsConnected) websocket.sendTXT(json);

```

Listing 9.2: Escaneo y serialización del espectro Wi-Fi

9.1.3. Portal cautivo y *buffer* offline

Al entrar en modo SoftAP, `startCaptivePortal()` inicia un servidor DNS que redirige todas las consultas a la IP del ESP32 y un servidor HTTP que sirve el formulario. El

manejador del formulario delega en `queueOrSendReg()`, que envía el registro o lo encola si no hay enlace (Listado 9.3).

```

1 void queueOrSendReg(const String& nombre, const String& apellido,
2                   const String& telefono, const String& apSsid) {
3     if (isWsConnected) {
4         String j = buildRegJson(nombre, apellido, telefono, apSsid); //
5             local (sendTXT pide String&)
6         websocket.sendTXT(j);
7     } else if (pendingCount < MAX_PENDING) {           // sin enlace:
8         encolar
9         pendingRegs[pendingCount++] = { nombre, apellido, telefono, apSsid
10            };
11     }
12 }
13 void flushPendingRegs() {                             // al reconectar:
14     reenviar
15     if (!isWsConnected || pendingCount == 0) return;
16     for (int i = 0; i < pendingCount; i++) {
17         String j = buildRegJson(pendingRegs[i].nombre, pendingRegs[i].
18             apellido,
19                               pendingRegs[i].telefono, pendingRegs[i].
20                               apSsid);
21         websocket.sendTXT(j); delay(30);
22     }
23     pendingCount = 0;
24 }

```

Listing 9.3: Envío con buffer offline (firmware)

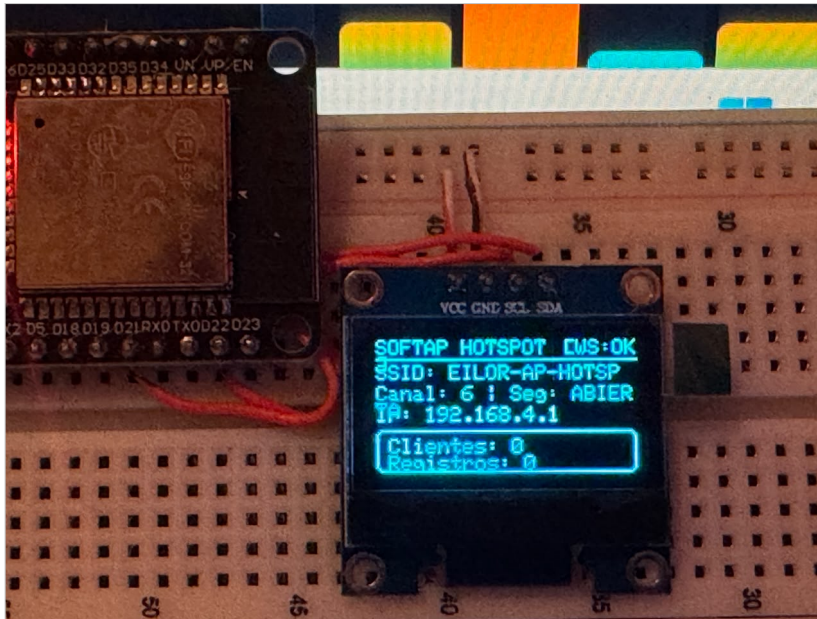


Figura 9.3: Pantalla OLED en modo SoftAP: SSID emitido, canal, IP del portal (192.168.4.1) y contadores de clientes y registros.

9.2. Implementación del servidor (Node.js)

9.2.1. Módulo de análisis de canales

El corazón analítico es `analyzeChannels()`, que calcula un puntaje de interferencia por canal ponderando el solapamiento de canales adyacentes (hasta 4 canales de distancia) y la intensidad de señal, y recomienda el canal más limpio del conjunto no solapado (Listado 9.4).

```

1 function analyzeChannels(wifiHeatmap) {
2   const scores = {};
3   for (let ch = 1; ch <= 13; ch++) {
4     let score = 0;
5     for (let other = 1; other <= 13; other++) {
6       const data = wifiHeatmap[other];
7       if (!data || !data.count) continue;
8       const dist = Math.abs(ch - other);
9       if (dist > 4) continue;                                // sin
        solapamiento significativo
10      const overlapWeight = 1 - dist / 5;                    // decae con la
        distancia
11      const rssiWeight = Math.max(0.2, Math.min(1, (data.maxRssi + 95)
        / 55));
12      score += data.count * overlapWeight * rssiWeight;
13    }
14    scores[ch] = Math.round(score * 10) / 10;

```

```

15 }
16 // ... determina bestChannel, worstChannel y recommendedChannel
    (1/6/11)
17 return { scores, bestChannel, worstChannel, recommendedChannel };
18 }

```

Listing 9.4: Puntaje de interferencia por canal (lib/analysis.js)

9.2.2. Detección de gemelo malicioso

La heurística agrupa por SSID y marca como sospechosos los que aparecen bajo más de un BSSID (Listado 9.5).

```

1 function detectEvilTwins(devices) {
2   const bySsid = new Map();
3   devices.forEach(d => {
4     if (d.type !== 'wifi') return;
5     const ssid = (d.ssid || '').trim();
6     if (!ssid || ssid === 'Oculto') return;
7     if (!bySsid.has(ssid)) bySsid.set(ssid, new Set());
8     bySsid.get(ssid).add((d.mac || '').toLowerCase());
9   });
10  const flagged = new Set();
11  bySsid.forEach(mac => { if (mac.size > 1) macs.forEach(m =>
12    flagged.add(m)); });
13  return flagged; // BSSIDs bajo el
    mismo SSID

```

Listing 9.5: Detección de evil twin (lib/analysis.js)

9.2.3. Identificación de fabricante (OUI)

El módulo lib/oui mantiene un catálogo de 123 prefijos y detecta MAC aleatorizadas por el segundo *nibble* del primer octeto (Listado 9.6).

```

1 function lookupVendor(mac) {
2   const clean = normalizeMac(mac);
3   const prefix = clean.slice(0, 6);
4   if (OUI_MAP[prefix]) return OUI_MAP[prefix];
5   const secondNibble = clean[1]; // MAC localmente
    administrada
6   if (['2', '6', 'A', 'E'].includes(secondNibble)) return 'MAC Aleatoria
    (privacidad)';
7   return 'Desconocido';
8 }

```

Listing 9.6: Búsqueda de fabricante y MAC aleatoria (lib/oui.js)

9.2.4. Autenticación por token

El módulo `lib/auth` implementa sesiones por token con expiración y un *middleware* `requireAuth` para proteger las rutas de control, además de `deviceAuthorized` para la ingesta (Listado 9.7).

```

1 function login(password) {
2   if (password !== PASSWORD) return null;
3   const token = crypto.randomBytes(24).toString('hex');
4   tokens.set(token, Date.now() + TTL_MS);           // TTL 24 h
5   return token;
6 }
7 function requireAuth(req, res, next) {
8   const token = req.headers['x-eilor-token'] || req.query.token || (
9     req.body && req.body.token);
10  if (isValidToken(token)) return next();
11  return res.status(401).json({ error: 'No autorizado. Inicia sesion.'
12    });
13 }
14 function deviceAuthorized(t) { return !DEVICE_TOKEN || t ===
15   DEVICE_TOKEN; }

```

Listing 9.7: Sesiones y middleware de autorización (lib/auth.js)

9.2.5. Ingesta y difusión en tiempo real

Al recibir un escaneo, `server.js` anota cada dispositivo con fabricante, MAC aleatoria y bandera de gemelo malicioso, recalcula el mapa de calor y difunde el resultado a los clientes (Listado 9.8).

```

1 function annotateDevices(list) {
2   return list.map(dev => ({
3     ...dev,
4     vendor: dev.vendor || lookupVendor(dev.mac),
5     randomMac: dev.randomMac !== undefined ? dev.randomMac :
6       isRandomMac(dev.mac),
7     evilTwin: evilTwinMacs.has((dev.mac || '').toLowerCase())
8   }));
9 }
10 // tras procesar el lote:
11 broadcast({ event: 'scan_update', heatmap: channelHeatmap,
12   latestBatch: annotateDevices(Array.from(activeDevices.
13     values())) });

```

Listing 9.8: Anotación y difusión (server.js)

9.3. Implementación del frontend (dashboard)

El cliente web establece la conexión WebSocket, solicita el estado inicial y despacha los eventos entrantes hacia los distintos renderizadores (mapa de calor, radar, tabla de dispositivos, registros y serie temporal). El Listado 9.9 ilustra el manejo del evento de escaneo.

```

1 if (data.event === 'initial_state' || data.event === 'scan_update') {
2   if (data.latestBatch) state.devices = data.latestBatch;
3   if (data.heatmap) { state.heatmap = data.heatmap; renderHeatmap(
4     data.heatmap); }
5   renderDevicesTable();
6   renderRadarBlips();
7   updateCounters();
8 }

```

Listing 9.9: Manejo de eventos en el cliente (public/app.js)



Figura 9.4: Dashboard web en operación: mapa de saturación espectral Wi-Fi (canales 1–13) y radar de proximidad BLE, alimentados en vivo por WebSocket.

Capítulo 10

Despliegue SaaS en la nube con Cloudflare

10.1. Modelo de entrega como servicio (SaaS)

EILOR se entrega bajo un modelo *Software as a Service*: el servidor Node.js se ejecuta de forma continua y se publica en Internet mediante un *túnel* de Cloudflare, de modo que tanto el dispositivo ESP32 como los clientes web acceden al sistema a través de un nombre de dominio público con cifrado TLS, sin exponer directamente la máquina ni abrir puertos entrantes en la red local. En la implementación de referencia, el sistema se publica en el dominio `esp.orellanadigitalco.trade` sobre el puerto 443 (WSS/HTTPS).

10.2. Arquitectura del túnel

Cloudflare Tunnel (`cloudflared`) establece una conexión saliente y persistente desde el servidor de origen hacia la red perimetral (*edge*) de Cloudflare. El tráfico de los usuarios llega primero al *edge*, donde se termina el TLS, y desde allí viaja por el túnel hasta el servidor local. La Figura 10.1 ilustra el modelo.

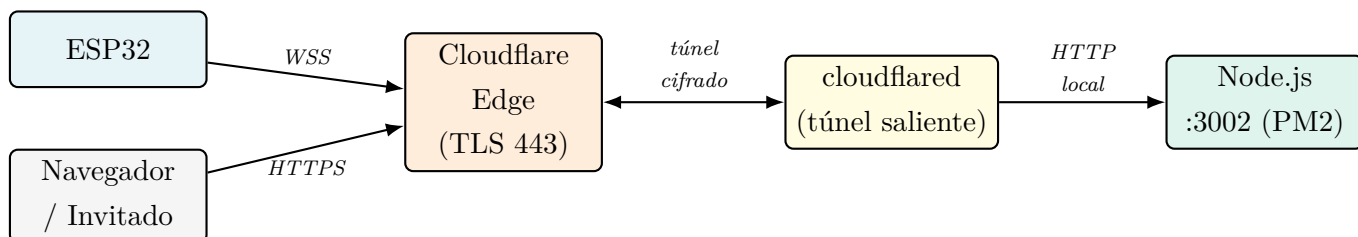


Figura 10.1: Publicación del servidor mediante Cloudflare Tunnel.

10.3. Ventajas de seguridad

- **Sin puertos entrantes:** el origen no expone puertos; el túnel es una conexión saliente, lo que reduce la superficie de ataque.
- **TLS gestionado:** el certificado y la terminación TLS los administra Cloudflare, garantizando cifrado extremo para WSS y HTTPS.
- **Ocultamiento de IP:** la dirección real del servidor no se publica en el DNS.
- **Controles de borde:** posibilidad de aplicar reglas de firewall, limitación de tasa y protección frente a denegación de servicio en el *edge*.

10.4. Configuración de referencia

El Listado 10.1 muestra una configuración típica de `cloudflared` con reglas de *ingress* que enrutan el hostname público al servidor local, habilitando explícitamente WebSocket.

```

1 tunnel: eilor-tunnel
2 credentials-file: /home/saul/.cloudflared/<UUID>.json
3
4 ingress:
5   - hostname: esp.orellanadigitalco.trade
6     service: http://localhost:3002
7     originRequest:
8       noTLSVerify: true
9       # WebSocket queda habilitado por defecto en cloudflared
10  - service: http_status:404

```

Listing 10.1: Configuración de Cloudflare Tunnel (config.yml)

El registro DNS asociado es un **CNAME** del hostname público hacia `<UUID>.cfargotunnel.com`, creado automáticamente por `cloudflared tunnel route dns`. Del lado del firmware, la conexión se establece con TLS mediante `websocket.beginSSL(WSS_SERVER_HOST, 443, "/")`, coherente con la terminación TLS en el *edge*.

10.5. Gestión del proceso con PM2

El servidor se ejecuta bajo el gestor de procesos PM2, que garantiza su reinicio automático ante fallos y su arranque tras un reinicio del sistema operativo. El Listado 10.2 resume los comandos de operación empleados.

```

1 pm2 start server.js --name eilor      # registrar el servicio
2 pm2 restart eilor --update-env        # recargar tras cambios de código

```

```
3 pm2 save                                # persistir para arranque
   automatico
4 pm2 logs eilor --lines 20              # inspeccionar registros
```

Listing 10.2: Operación del servicio con PM2

En conjunto, Cloudflare Tunnel y PM2 conforman la capa de *entrega y operación* del SaaS: alta disponibilidad del proceso, publicación segura sin exposición directa y acceso cifrado uniforme para el hardware y los usuarios.

Capítulo 11

Plataforma web: interfaz, uso y comunicación

Este capítulo documenta la plataforma de análisis y control de EILOR desde la perspectiva de su uso: cómo se ingresa, qué opciones ofrece, cómo está construida su interfaz y de qué manera se comunica con el dispositivo ESP32. La instancia de referencia se encuentra publicada en <https://esp.orellanadigitalco.trade/>, servida a través del túnel de Cloudflare descrito en el Capítulo 10.

11.1. Acceso a la plataforma

La plataforma se abre desde cualquier navegador moderno en la dirección pública <https://esp.orellanadigitalco.trade/> o, en despliegues locales, en `http://localhost:3002`. Al cargar, el *dashboard* muestra de inmediato —en modo de *solo lectura*— el estado del ESP32, el modo activo, los contadores de dispositivos y paquetes, el mapa de saturación espectral y el radar BLE, sin exigir credenciales. Este acceso público de solo lectura permite observar el espectro sin poder emitir comandos.

11.1.1. Inicio de sesión

Para habilitar las *acciones de control* —cambiar de modo, desplegar el SoftAP, limpiar datos, listar o exportar registros— se requiere iniciar sesión. Al pulsar *Iniciar Sesión* aparece el formulario de la Figura 11.1, que solicita la contraseña configurada en la variable de entorno `EILOR_PASSWORD`. Si la contraseña es correcta, el servidor emite un *token de sesión* (cadena hexadecimal de 48 caracteres, con vigencia de 24 h) que el navegador conserva y adjunta en la cabecera `x-eilor-token` de cada petición protegida. Una contraseña incorrecta devuelve HTTP 401 y la sesión no se habilita.

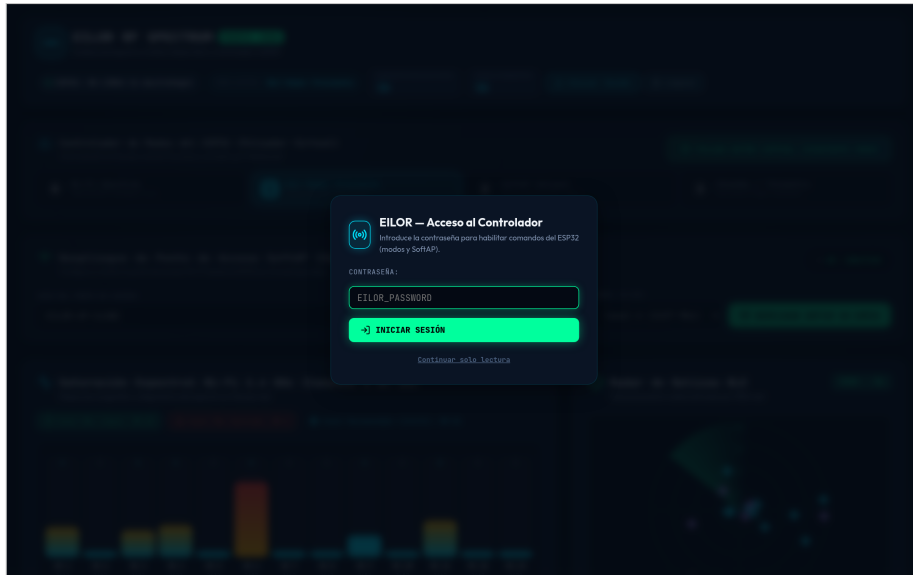


Figura 11.1: Formulario de inicio de sesión de la plataforma (habilita los comandos del ESP32).

11.2. Estructura y opciones de la interfaz

El *dashboard* organiza sus funciones en secciones apiladas verticalmente. La Figura 11.2 muestra la cabecera de estado, con el indicador de conexión del ESP32, el modo activo y los contadores globales, además de los botones de sesión y de reinicio de datos.

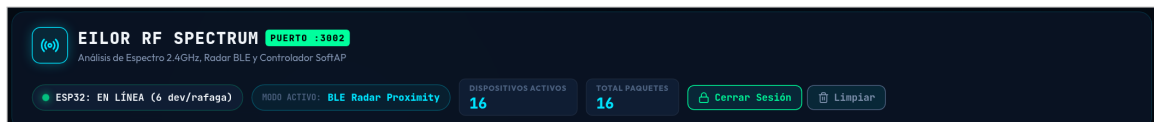


Figura 11.2: Cabecera de estado: enlace del ESP32, modo activo y contadores de dispositivos y paquetes.

11.2.1. Controlador de modos

La sección *Controlador de Modos* (Figura 11.3) actúa como un *pulsador virtual*: replica en la web el botón físico del dispositivo. Permite seleccionar directamente cualquiera de los cuatro modos —(0) Espectro Wi-Fi, (1) Radar BLE, (2) SoftAP con portal cautivo y (3) Standby— o avanzar al siguiente. Cada cambio se emite por WebSocket y el ESP32 lo aplica en tiempo real.

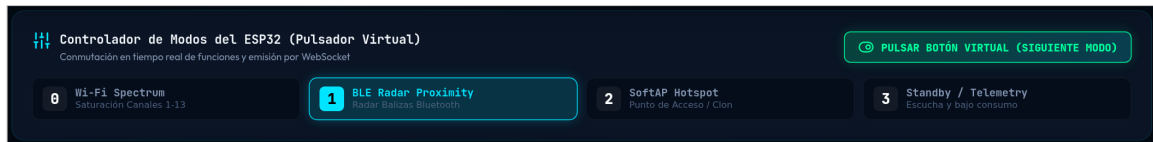


Figura 11.3: Controlador de modos: conmutación remota de las cuatro funciones del ESP32.

11.2.2. Despliegue del punto de acceso (SoftAP)

En la sección *Despliegue de Punto de Acceso* (Figura 11.4) se define el SSID y el canal (1–13) del punto de acceso propio y se pulsa *Desplegar SoftAP en ESP32*. El dispositivo pasa al modo 2, emite la red y levanta el portal cautivo de registro. Esta funcionalidad está concebida exclusivamente para redes propias, con un nombre identificable y aviso de registro, conforme a las consideraciones éticas del Capítulo 17.

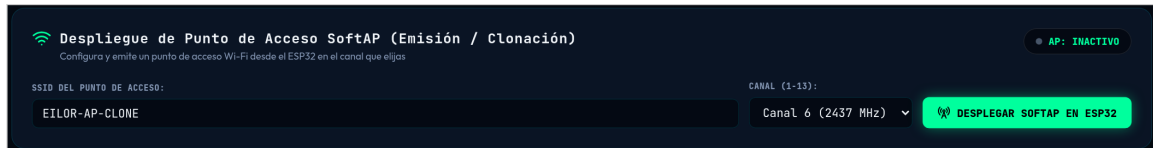


Figura 11.4: Panel de despliegue del SoftAP: selección de SSID y canal del punto de acceso propio.

Cuando un invitado se conecta a la red emitida, su dispositivo es redirigido automáticamente por el portal cautivo al formulario de registro (Figura 11.5), que solicita nombre, apellido y teléfono e incluye un aviso explícito sobre la recolección de datos.

VISTA PREVIA — así se ve el portal en el móvil. El real lo sirve el ESP32 en Modo 2. Aquí el envío es solo demostración.

WiFi de Invitados

Regístrate para conectarte

Nombre
Tu nombre

Apellido
Tu apellido

Teléfono
Ej. 099 123 4567

Conectarme

Red de invitados. Al registrarte aceptas dejar tus datos de contacto para el acceso.

Figura 11.5: Formulario del portal cautivo tal como lo ve el invitado en su teléfono (vista previa).

11.2.3. Análisis del espectro y radar BLE

El núcleo analítico se visualiza en dos paneles contiguos (Figura 11.6): a la izquierda, el mapa de *saturación espectral* de los canales 1 al 13, con el canal más limpio, el más saturado y el canal recomendado del conjunto no solapado 1/6/11; a la derecha, el *radar de proximidad BLE*, que ubica las balizas de forma radial según su RSSI. Ambos se actualizan en vivo con cada ciclo de escaneo.

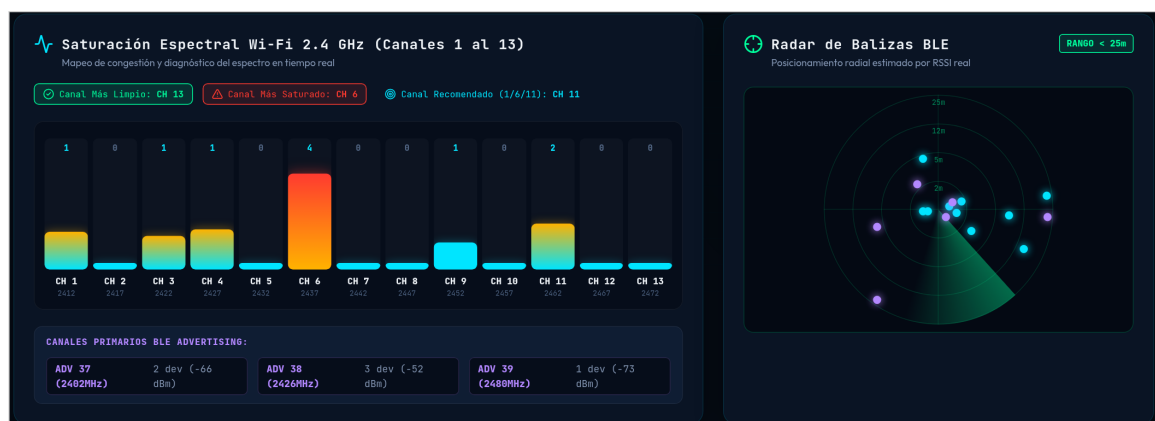


Figura 11.6: Saturación espectral Wi-Fi (canales 1–13) y radar de proximidad BLE en tiempo real.

11.2.4. Telemetría y flujo de comandos

El *Panel de Telemetría* (Figura 11.7) presenta una consola en vivo con las tramas WebSocket, los eventos de conexión y las respuestas de comandos, acompañada de una gráfica de serie temporal (dispositivos activos y paquetes por minuto). Es la herramienta principal de diagnóstico del enlace en la demostración.



Figura 11.7: Panel de telemetría: consola de tramas WebSocket y serie temporal de actividad.

11.2.5. Inventario de dispositivos

La tabla de *Registro de Dispositivos* (Figura 11.8) enumera cada dispositivo detectado con su tipo, nombre/SSID, dirección MAC/BSSID, fabricante (por OUI), canal, RSSI, distancia estimada y tipo de seguridad. Los puntos de acceso sospechosos de *gemelo malicioso* se marcan con una etiqueta «Gemelo», y las direcciones aleatorizadas se rotulan como «MAC Aleatoria (privacidad)». La tabla admite filtrado por tipo (Wi-Fi/Bluetooth) y búsqueda por SSID o MAC.

Tipo	Nombre / SSID / Dispositivo	Dirección MAC / BSSID	Fabricante (OUI)	Canal	Potencia RSSI	Distancia Est.	Seguridad / Tipo	Acción Rápida
Wi-Fi 2.4G	Cafe-Centro	A8:5C:2C:11:22:33	Apple	CH 6	-46 dBm	~1.1 m	3	Crear AP
Wi-Fi 2.4G	Oficina-2G	D8:5D:4C:44:55:66	TP-Link	CH 1	-63 dBm	~5.6 m	4	Crear AP
Wi-Fi 2.4G	Tienda_Norte	50:C7:BF:10:20:30	TP-Link	CH 6	-71 dBm	~12.1 m	3	Crear AP
Wi-Fi 2.4G	CasaWifi (1) Gemelo	14:CC:20:AA:00:01	TP-Link	CH 6	-55 dBm	~2.6 m	3	Crear AP
Wi-Fi 2.4G	CasaWifi (1) Gemelo	9E:3A:11:AA:00:02	MAC Aleatoria (privacidad)	CH 6	-58 dBm	~3.5 m	0	Crear AP
Wi-Fi 2.4G	Biblioteca	00:24:D7:33:44:55	Intel	CH 11	-80 dBm	~28.7 m	4	Crear AP

Figura 11.8: Inventario de dispositivos con fabricante por OUI, marcado de gemelo malicioso y de MAC aleatorizada.

11.2.6. Registros del portal cautivo

La sección *Registros del Portal Cautivo* (Figura 11.9) lista en tiempo real las altas de invitados (nombre, apellido, teléfono, red y fecha). Desde allí se puede *Exportar CSV* (con BOM UTF-8 para su apertura correcta en hojas de cálculo) o *Borrar* los registros; ambas acciones exigen sesión autenticada.

FECHA / HORA	NOMBRE	APELLIDO	TELÉFONO	RED (SSID)
9/23/2026, 6:25:10 AM	Jorge	Velez	096 555 1212	Cafe-EILOR
9/23/2026, 6:25:10 AM	Lucia	Andrade	097 222 3344	Cafe-EILOR
9/23/2026, 6:25:10 AM	Carlos	Mena	098 765 4321	Cafe-EILOR
9/23/2026, 6:25:10 AM	Ana	Torres	099 111 2233	Cafe-EILOR

Figura 11.9: Registros del portal cautivo en el *dashboard*, con exportación restringida a la sesión.

11.3. Diseño de la interfaz web

El frontend se construyó sin *frameworks*, con HTML, CSS y JavaScript nativos, priorizando la transparencia y la ligereza. El diseño visual adopta una estética de «tablero táctico» oscuro, definida mediante *variables CSS* (tokens de color y tipografía) que unifican la apariencia. La Tabla 11.1 resume las decisiones de diseño y los archivos que las materializan.

Tabla 11.1: Composición del frontend de la plataforma

Aspecto	Archivo	Descripción
Estructura	<code>public/index.html</code>	Secciones del <i>dashboard</i> y superposición de <i>login</i> (419 líneas)
Lógica	<code>public/app.js</code>	Cliente WebSocket, renderizadores y control de sesión (913 líneas)
Estilos	<code>public/styles.css</code>	Tokens de color, tipografía y componentes «tácticos» (1272 líneas)
Portal	<code>public/portal-preview.html</code>	Vista previa del formulario del portal cautivo
Tipografía	Google Fonts	<i>Outfit</i> (display) y <i>JetBrains Mono</i> (monoespaciada)
Iconografía	Lucide	Iconos vectoriales ligeros vía CDN

El cliente mantiene un objeto de estado en memoria (**state**) que consolida los dispositivos, el mapa de calor, la serie temporal y los registros; ante cada evento WebSocket se actualiza el estado y se invoca únicamente al renderizador afectado (mapa de calor,

radar, tabla, registros o serie temporal), evitando redibujar toda la página. El token de sesión se guarda en `localStorage` para no repetir el inicio de sesión entre recargas.

11.4. Comunicación entre la web, el servidor y el ESP32

La plataforma se comunica con el dispositivo de forma *indirecta*: nunca habla directamente con el ESP32, sino a través del servidor Node.js, que actúa como concentrador. Existen dos canales complementarios:

- **WebSocket (tiempo real)**: tanto el ESP32 como los navegadores mantienen una conexión WebSocket persistente con el servidor. El ESP32 envía sus escaneos, *heartbeats* y registros de invitados; el servidor los procesa y *difunde (push)* los resultados a todos los navegadores conectados. En sentido inverso, los comandos de la web (cambio de modo, despliegue de SoftAP) viajan al servidor, que los reenvía al ESP32 por el mismo canal.
- **API REST (consultas y control)**: la web dispone además de catorce *endpoints* HTTP (Tabla 6.2) para inicio de sesión, consultas de estado, control y gestión de registros, empleados en las operaciones que no requieren tiempo real.

La Figura 11.10 ilustra el flujo de un comando de control emitido desde la web hasta el ESP32 y la difusión de vuelta del nuevo estado.

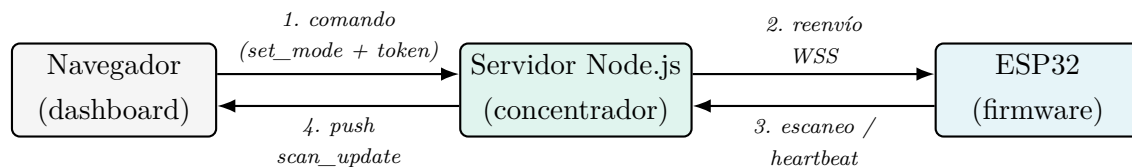


Figura 11.10: Flujo de comunicación web ↔ servidor ↔ ESP32 (el servidor concentra ambos sentidos).

Las acciones de control validan el *token de sesión* antes de reenviarse al dispositivo; un comando WebSocket sin token válido recibe un evento `auth_error` y no se ejecuta. La ingesta de escaneos, a su vez, puede restringirse con un *token de dispositivo* opcional (`EILOR_DEVICE_TOKEN`). El detalle de los mensajes WebSocket y de los *endpoints* REST se encuentra en la Sección 6 del diseño detallado, y la publicación segura del servicio mediante el túnel de Cloudflare, en el Capítulo 10.

11.5. Vista pública en producción

La Figura 11.11 muestra la plataforma tal como se sirve en el dominio público <https://esp.orellanadigitalco.trade/> sobre HTTPS/WSS, evidenciando el despliegue real de extremo a extremo (dispositivo, túnel, servidor y cliente).

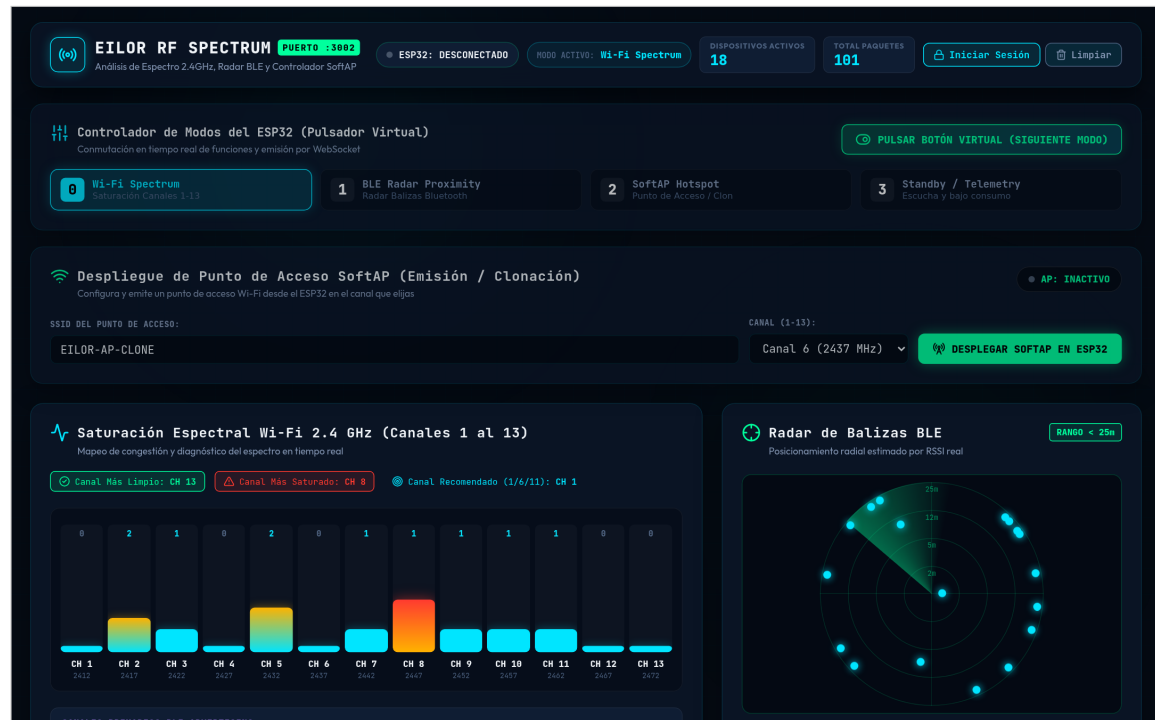


Figura 11.11: Plataforma EILOR publicada en esp.orellanadigitalco.trade a través del túnel de Cloudflare.

Capítulo 12

Pruebas de validación y seguridad

12.1. Estrategia de pruebas

La validación combinó *pruebas funcionales* de extremo a extremo (ingesta, análisis, persistencia y registro) con *pruebas de seguridad* centradas en el control de acceso. Las pruebas se ejecutaron mediante un *banco automatizado* en Node.js que levanta instancias aisladas del servidor (sin datos reales), inyecta escaneos y registros simulados por REST y por WebSocket, verifica las respuestas y reinicia el proceso para comprobar la persistencia. El banco reporta cada caso con su resultado observado y una medición de latencia; la totalidad de los casos aquí documentados se superó de forma reproducible (véase la Figura 12.1 en la Sección 12.6).

12.2. Casos de prueba funcionales

La Tabla 12.1 resume los casos ejecutados con su resultado observado.

Tabla 12.1: Casos de prueba funcionales y resultados

ID	Descripción	Esperado	Estado
CP01	Ingesta de escaneo Wi-Fi (3 dispositivos)	<code>received:3</code>	OK
CP02	Fabricante por OUI (A85C2C, D85D4C)	Apple, TP-Link	OK
CP03	Detección de MAC aleatoria	Marca privacidad	OK
CP04	Evil twin (2 BSSID, SSID "CasaWifi")	Ambos marcados	OK
CP05	Análisis de canal (best/worst/rec.)	Salida coherente	OK
CP06	Serie temporal (muestra por minuto)	Muestras acumuladas	OK
CP07	Persistencia y restauración tras reinicio	Estado recuperado	OK
CP08	Alta de invitado por el portal	<code>success + total</code>	OK
CP09	Exportación CSV con BOM UTF-8	CSV con acentos	OK
CP10	Borrado de registros (con sesión)	<code>cleared</code>	OK

12.3. Casos de prueba de seguridad

Las pruebas de seguridad verificaron que las acciones sensibles exigen las credenciales adecuadas. La Tabla 12.2 presenta los resultados; los códigos de estado HTTP obtenidos confirman el correcto funcionamiento del control de acceso.

Tabla 12.2: Casos de prueba de seguridad (control de acceso)

ID	Escenario	Esperado	Obtenido
SEC01	POST /api/mode sin token de sesión	401	401
SEC02	POST /api/mode con token válido	200	200
SEC03	GET /api/registrations sin sesión	401	401
SEC04	GET /api/registrations con sesión	200	200
SEC05	Ingesta con EILOR_DEVICE_TOKEN y sin token	401	401
SEC06	Ingesta con token de dispositivo correcto	200	200
SEC07	Login con contraseña incorrecta	401	401

12.4. Análisis de amenazas y mitigaciones

La Tabla 12.3 relaciona las principales amenazas con los controles implementados, siguiendo el enfoque de buenas prácticas de OWASP (OWASP Foundation, [2021](#)).

Tabla 12.3: Amenazas y controles implementados

Amenaza	Control en EILOR
Comandos de control no autorizados	Token de sesión (TTL 24 h) y <code>requireAuth</code> en rutas de control
Inyección de datos falsos por terceros	Token de dispositivo opcional (<code>deviceAuthorized</code>) en la ingesta
Suplantación de red (evil twin)	Detección y marcado de SSID con múltiples BSSID
Rastreo por MAC	Detección y señalización de MAC aleatorizadas
Exposición del servidor	Publicación por túnel Cloudflare sin puertos entrantes y TLS en el borde
Fuga de datos de invitados	Exportación restringida a sesión; minimización de campos recolectados

12.5. Métricas de calidad y desempeño observadas

La Tabla 12.4 consolida las métricas medidas durante las pruebas.

Tabla 12.4: Métricas de desempeño observadas

Métrica	Valor
Periodo de ciclo de escaneo	~3 s
Periodo de <i>heartbeat</i>	3 s
Umbral de <i>watchdog</i> de desconexión	15 s
Frecuencia de muestreo de serie temporal	1 muestra/min (720 muestras $\approx$ 12 h)
Frecuencia de <i>snapshot</i> de estado	30 s
Catálogo OUI	123 prefijos
Latencia ingesta $\rightarrow$ <i>push</i> (mediana / P95)	2.5 / 3.2 ms
Casos funcionales superados	10/10 (100 %)
Casos de seguridad superados	7/7 (100 %)

12.6. Evidencia de ejecución

La Figura 12.1 reproduce la salida del banco automatizado. Además de los casos de las Tablas 12.1 y 12.2, el banco ejercita rutas adicionales —registro por WebSocket, rechazo de comandos WebSocket sin sesión, invalidación de token tras *logout* y registro con token de dispositivo erróneo— alcanzando 20/20 verificaciones superadas.

```

=== EILOR :: banco de pruebas === 2026-09-23T11:22:11.233Z
Node v24.20.0
[OK] CP01 Ingesta de escaneo Wi-Fi (3 dispositivos) -> HTTP 200, received=3
[OK] CP02 Fabricante por OUI (A85C2C, D85D4C) -> Apple, TP-Link
[OK] CP03 Deteccion de MAC aleatoria (DA:..) -> randomMac=true, vendor="MAC Aleatoria (priva
[OK] CP04 Evil twin (2 BSSID, SSID "CasaWifi") -> 2 BSSID marcados
[OK] CP05 Analisis de canal (best/worst/rec.) -> best=12 worst=6 rec=11 score6=2.6
... esperando 62 s al muestreador de serie temporal
[OK] CP06 Serie temporal (muestra por minuto) -> 1 muestra(s); ultima: active=5, rate=5
[OK] CP08 Alta de invitado por el portal (HTTP) -> HTTP 200, total=1
[OK] SEC07 Login con contraseña incorrecta -> HTTP 401
login correcto -> HTTP 200, token de 48 caracteres hex
[OK] SEC01 POST /api/mode sin token de sesión -> HTTP 401
[OK] SEC02 POST /api/mode con token valido -> HTTP 200, modo=BLE Radar Proximity
[OK] SEC03 GET /api/registrations sin sesión -> HTTP 401
[OK] SEC04 GET /api/registrations con sesión -> HTTP 200, total=2
[OK] CP09 Exportacion CSV con BOM UTF-8 -> BOM=true, contiene "Núñez"=true
[OK] SEC08 Comando WS set_mode sin token -> evento auth_error: "Sesión requerida para enviar
[OK] SEC09 Token invalidado tras logout -> HTTP 401
LAT ingesta REST -> push WS (n=100): min=0.90 p50=2.49 p95=3.16 max=5.42 avg=2.52 ms
LAT respuesta REST /api/scan (n=100): min=0.85 p50=2.39 p95=3.04 max=5.33 avg=2.41 ms
[OK] CP07 Persistencia y restauracion tras reinicio -> 5 dispositivos, 1 muestras, 2 registr
[OK] CP10 Borrado de registros (con sesión) -> HTTP 200, {"status":"cleared"}
[OK] SEC05 Ingesta con EILOR_DEVICE_TOKEN y sin token -> HTTP 401
[OK] SEC06 Ingesta con token de dispositivo correcto -> HTTP 200, received=3
[OK] SEC10 Registro HTTP con token de dispositivo erróneo -> HTTP 401
=== RESUMEN: 20/20 casos superados ===

```

Figura 12.1: Salida de consola del banco de pruebas automatizado: 20/20 casos superados, con los códigos de control de acceso (401/200) y las latencias medidas.



Registros del Portal Cautivo (Invitados)
Alfas enviados desde el formulario del punto de acceso en tiempo real

TOTAL REGISTROS: 4

[Exportar CSV](#) [Borrar](#)

FECHA / HORA	NOMBRE	APELLIDO	TELÉFONO	RED (SSID)
9/23/2026, 6:25:10 AM	Jorge	Vélez	096 555 1212	Cafe-EILOR
9/23/2026, 6:25:10 AM	Lucia	Andrade	097 222 3344	Cafe-EILOR
9/23/2026, 6:25:10 AM	Carlos	Mena	098 765 4321	Cafe-EILOR
9/23/2026, 6:25:10 AM	Ana	Torres	099 111 2233	Cafe-EILOR

Figura 12.2: Registros de invitados visibles en el *dashboard* en tiempo real, con exportación a CSV restringida a la sesión autenticada.

Capítulo 13

Evaluación de desempeño

13.1. Metodología de evaluación

La evaluación de desempeño contrastó los indicadores medidos frente a los criterios de logro fijados en la matriz SMART (Tabla 4.1). Se consideraron tres dimensiones: *capacidad de captura* (dispositivos y canales procesados), *latencia* (periodos de ciclo y difusión) y *robustez* (persistencia y recuperación).

13.2. Cumplimiento de objetivos

La Tabla 13.1 evalúa cada objetivo específico frente a su indicador.

Tabla 13.1: Cumplimiento de objetivos frente a los indicadores SMART

OE	Indicador	Meta	Resultado
OE1	Canales y latencia de ciclo	13+3; ≤ 3 s	13+3; ~ 3 s
OE2	Puntaje y canal recomendado	13 canales; 1/6/11	Cumplido
OE3	Catálogo OUI	≥ 120	123 prefijos
OE4	Control autenticado	401/200	401/200
OE5	Registros entregados	100 % con enlace	Con <i>buffer</i>
OE6	Casos superados	≥ 90 %	100 %

13.3. Análisis de latencia y capacidad

El ciclo de escaneo de ~ 3 s ofrece un compromiso adecuado entre frescura de la información y carga de la radio del ESP32, que debe multiplexar Wi-Fi y BLE en modos conmutables. La difusión por WebSocket agrega latencias despreciables frente al ciclo de escaneo, dado el modelo de *push* del servidor: en una medición automatizada

sobre 100 ingestas consecutivas, el tiempo de extremo a extremo entre la recepción de un lote (`POST /api/scan`) y la difusión del evento `scan_update` a un cliente WebSocket presentó una mediana de **2.5 ms** y un percentil 95 de **3.2 ms** (máximo 5.4 ms). En el lado del servidor, el modelo de E/S no bloqueante de Node.js (OpenJS Foundation, 2024) permite atender simultáneamente el socket del dispositivo y múltiples clientes del dashboard sin degradación perceptible en el rango de prueba. La Tabla 13.2 resume la distribución medida.

Tabla 13.2: Latencia de procesamiento y difusión (100 ingestas consecutivas, servidor local)

Trayecto medido	Mín.	Mediana	P95	Máx.
Respuesta REST <code>POST /api/scan</code>	0.85 ms	2.39 ms	3.04 ms	5.33 ms
Ingesta REST $\rightarrow$ <i>push</i> WebSocket (extremo a extremo)	0.90 ms	2.49 ms	3.16 ms	5.42 ms

Estos valores corresponden al procesamiento del servidor (anotación de fabricante, recálculo del mapa de calor, detección de gemelos maliciosos y difusión) y excluyen el ciclo de escaneo del dispositivo y la latencia de la red o del túnel; confirman que la lógica de agregación no constituye un cuello de botella frente al periodo de escaneo de 3 s.

13.4. Robustez y disponibilidad

La prueba CP07 confirmó que, tras un reinicio ordenado, el sistema recupera el estado persistido (dispositivos, serie temporal y registros), evidenciado por el mensaje de restauración del módulo `store`. Combinado con la gestión de PM2 —que reinicia el proceso ante fallos— el sistema alcanza una disponibilidad operativa alta para un prototipo. El *watchdog* de 15 s detecta y notifica la pérdida del enlace con el dispositivo, actualizando el estado en el dashboard.

13.5. Comparación con los objetivos planteados

La Figura 13.1 resume gráficamente el nivel de cumplimiento por objetivo, todos en o por encima de la meta.

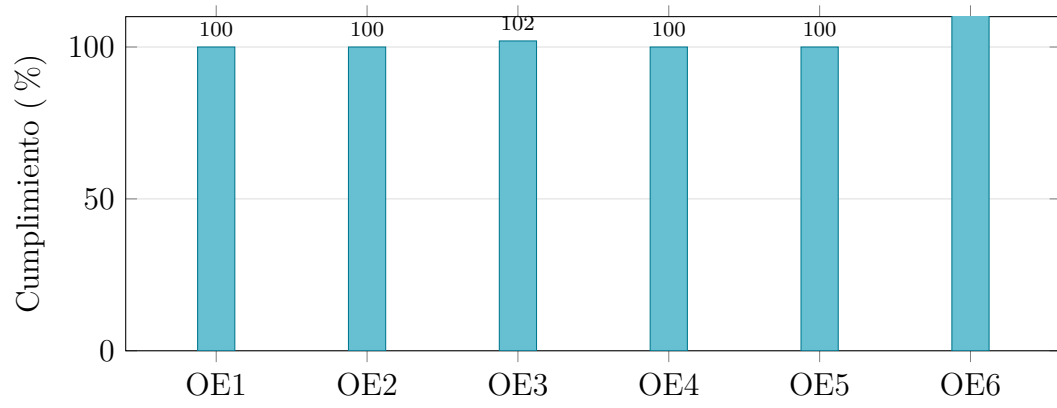


Figura 13.1: Nivel de cumplimiento por objetivo específico (100 % = meta).

Capítulo 14

Análisis económico y ambiental

14.1. Estimación de costos

El costo total del proyecto se descompone en hardware, desarrollo (horas-persona) y operación en la nube. La Tabla 14.1 presenta la estimación.

Tabla 14.1: Estimación de costos del proyecto

Rubro	Cant.	Costo unit. (USD)	Subtotal (USD)
Hardware (BOM, Tabla 7.2)	1	24.20	24.20
Desarrollo firmware (h)	40	8.00	320.00
Desarrollo servidor y frontend (h)	60	8.00	480.00
Pruebas y documentación (h)	30	8.00	240.00
Operación en la nube (mes)	12	0.00	0.00
Costo total estimado			1 064.20

El costo operativo se mantiene en cero gracias al uso del nivel gratuito de Cloudflare Tunnel y a la ejecución del servidor en un equipo propio, lo que hace al sistema especialmente atractivo para pequeños negocios. El costo marginal de replicar una unidad de hardware adicional, incluida la alimentación autónoma, es de aproximadamente USD 24.20.

14.2. Curva S del proyecto

La Figura 14.1 presenta la curva S del avance acumulado planificado frente al real a lo largo de las 16 semanas de la macro actividad, expresado como porcentaje de avance ponderado por el peso de cada entrega (20 %, 30 %, 25 % y 25 %).

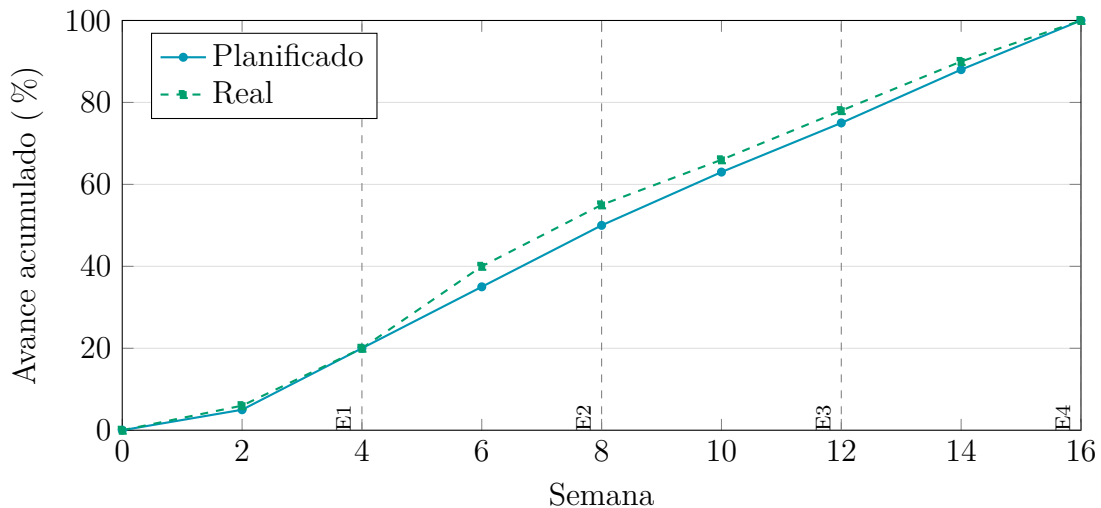


Figura 14.1: Curva S: avance acumulado planificado vs. real.

14.3. Análisis de impacto ambiental

El impacto ambiental de EILOR es reducido y, en balance, potencialmente positivo:

- **Consumo energético:** el ESP32 opera en el orden de cientos de miliamperios; su huella energética es marginal frente a equipos de instrumentación tradicionales.
- **Ciclo de vida del hardware:** el uso de una placa reutilizable y componentes estándar favorece la reparación y la reutilización, en línea con principios de electrónica sostenible.
- **Eficiencia del espectro:** al recomendar canales menos congestionados, el sistema contribuye indirectamente a un uso más eficiente del recurso radioeléctrico, reduciendo retransmisiones y, con ello, el consumo energético agregado de las redes cercanas.
- **Software sin dependencias pesadas:** la persistencia en archivos y la ausencia de bases de datos y servicios adicionales minimizan el cómputo requerido en operación.

14.4. Relación costo-beneficio

Con una inversión de hardware inferior a USD 25 por unidad y costo operativo nulo, EILOR entrega capacidades —diagnóstico de espectro, detección de amenazas y gestión de invitados— que en soluciones comerciales requieren equipos de costo muy superior. Para un pequeño negocio, el retorno se materializa tanto en la mejora del servicio Wi-Fi como en el valor de la base de contactos de invitados obtenida de forma consentida.

Capítulo 15

Optimización y escalabilidad

15.1. Oportunidades de optimización identificadas

A partir de la evaluación de desempeño se identificaron mejoras concretas, resumidas en la Tabla 15.1, clasificadas por su impacto y esfuerzo.

Tabla 15.1: Oportunidades de optimización

Mejora	Impacto	Esfuerzo
Migración a base de datos relacional (SQLite/PostgreSQL)	Alto	Medio
Cifrado de contraseñas con <i>hashing</i> (bcrypt)	Alto	Bajo
Limitación de tasa (<i>rate limiting</i>) en el login	Medio	Bajo
HTTPS en el portal cautivo del ESP32	Medio	Alto
Compresión de tramas WebSocket	Bajo	Bajo
Soporte multi-dispositivo (varios ESP32)	Alto	Medio
Panel de administración de invitados con filtros	Medio	Medio

15.2. Plan de escalabilidad

15.2.1. Escalabilidad de datos

El desacoplamiento del módulo `lib/store` (Capítulo 8) permite sustituir la persistencia en archivos por un motor relacional sin modificar la lógica de negocio. Para retención prolongada y consultas analíticas, PostgreSQL con índices por marca temporal es la opción recomendada.

15.2.2. Escalabilidad de dispositivos

El modelo actual asume un dispositivo principal identificado por `deviceId`. Para soportar múltiples unidades, basta con particionar el estado por `deviceId` y extender el mapa de calor y la tabla de dispositivos con dicha dimensión, manteniendo una vista agregada por sede.

15.2.3. Escalabilidad de servicio

En caso de un número elevado de clientes del dashboard, la difusión WebSocket puede escalar horizontalmente introduciendo un *message broker* (por ejemplo, Redis *pub/sub*) que replique los eventos entre varias instancias del servidor tras un balanceador. Cloudflare, en el borde, aporta protección frente a picos de tráfico y denegación de servicio.

15.2.4. Endurecimiento de seguridad

Las siguientes acciones elevan la postura de seguridad para producción: almacenar contraseñas con *hashing* y sal (bcrypt), aplicar limitación de intentos en el *login*, rotar el token de dispositivo, y registrar auditoría de accesos. Estas mejoras son compatibles con la arquitectura actual y de bajo esfuerzo de integración.

15.3. Hoja de ruta

La Figura 15.1 propone una hoja de ruta por fases.

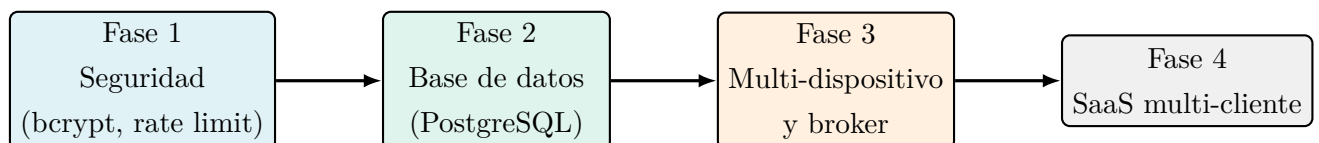


Figura 15.1: Hoja de ruta de evolución del sistema.

Capítulo 16

Manuales de usuario y técnico

16.1. Manual de usuario

El recorrido visual completo de la plataforma —con capturas de cada sección— se presenta en el Capítulo 11. Esta sección resume el uso operativo.

16.1.1. Requisitos previos y preparación

Antes de iniciar una sesión se debe disponer de la placa ESP32 con la pantalla OLED y el pulsador conectados, una fuente USB estable y un equipo con navegador moderno. Para la primera puesta en marcha se recomienda utilizar el cable USB del computador; para operación autónoma se utiliza el portapilas con salida USB regulada descrito en el Capítulo 7. No se deben conectar simultáneamente ambas fuentes. El operador debe comprobar que la antena de la placa no quede cubierta por metal y que el pulsador pueda accionarse sin abrir la carcasa.

16.1.2. Secuencia de arranque del ESP32

La secuencia normal de uso es la siguiente:

1. Conectar una única fuente de alimentación y esperar el mensaje inicial en la OLED.
2. El ESP32 intenta asociarse a la red Wi-Fi configurada y abre el WebSocket seguro con el servidor.
3. Confirmar en la cabecera del dashboard que el dispositivo figura como *En línea*; en la OLED debe aparecer `[WS:OK]`.
4. Seleccionar el modo desde el dashboard o pulsar brevemente el botón para avanzar al siguiente modo.

5. Mantener el dispositivo en una ubicación despejada, evitando acercarlo a routers o fuentes de ruido si se desea una medición representativa.

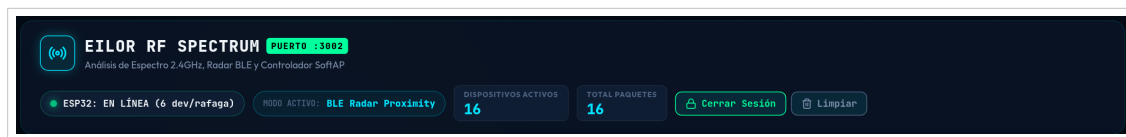


Figura 16.1: Indicadores de operación: conexión del ESP32, modo activo y contadores globales.

16.1.3. Acceso al dashboard

El tablero de mando se abre desde un navegador en la dirección publicada (por ejemplo, <https://esp.orellanadigitalco.trade/>) o localmente en <http://localhost:3002>. La interfaz muestra, en tiempo real, el estado del ESP32, el modo activo y los contadores de dispositivos y paquetes.

16.1.4. Inicio de sesión

Para ejecutar acciones de control (cambiar de modo, desplegar SoftAP, limpiar datos o ver registros) se requiere iniciar sesión con el botón *Iniciar Sesión* e introducir la contraseña configurada. Sin sesión, el sistema opera en modo de solo lectura.

16.1.5. Modos del dispositivo

Los cuatro modos se conmutan desde el panel de control o con el pulsador físico: (0) Espectro Wi-Fi, (1) Radar BLE, (2) SoftAP con portal cautivo y (3) Standby.

Modo 0 — Espectro Wi-Fi.

Recorre los canales 1–13 y muestra el total de redes, el canal más saturado y una recomendación. Es el modo apropiado para diagnosticar interferencia de acceso.

Modo 1 — Radar BLE.

Escucha las balizas de advertising y resume cantidad, nombre visible y RSSI del dispositivo más cercano. La ausencia de nombre no implica ausencia de dispositivo.

Modo 2 — SoftAP.

Crea la red propia de invitados y atiende el portal cautivo. Solo debe emplearse en redes autorizadas y con un SSID claramente identificable.

Modo 3 — Standby/telemetría.

Mantiene el enlace y envía estado, dirección IP y telemetría sin iniciar un ciclo de escaneo intensivo.

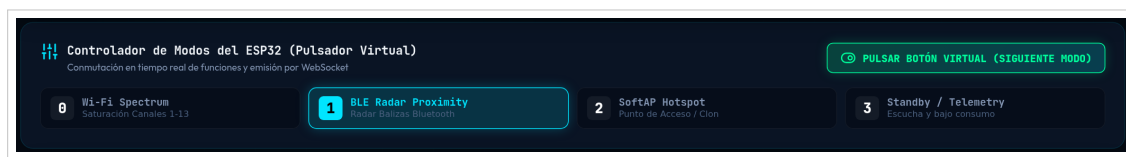


Figura 16.2: Selector de los cuatro modos disponibles para el ESP32.

En el modo Wi-Fi conviene esperar al menos un ciclo completo antes de interpretar la gráfica. En el radar BLE se deben comparar varias muestras porque el RSSI cambia con la orientación y el movimiento. El botón físico usa lógica activa en bajo: cada pulsación válida cambia de modo y el antirrebote evita cambios duplicados.

16.1.6. Lectura de espectro y radar

La vista de espectro permite localizar canales con mayor ocupación y comparar la distribución de puntos de acceso. La recomendación de canal es una ayuda heurística, no una garantía de rendimiento: debe contrastarse con el ancho de canal y la ubicación física del router. La vista radar muestra dispositivos BLE detectados, su RSSI y la última actualización recibida.



Figura 16.3: Uso del mapa de saturación Wi-Fi y del radar BLE para observar el entorno de 2.4 GHz.

16.1.7. Despliegue del punto de acceso y portal

En la sección SoftAP se define el SSID y el canal, y se pulsa *Desplegar SoftAP*. Al conectarse un invitado a esa red, aparece automáticamente el portal cautivo con el formulario de registro.

El procedimiento recomendado es: (1) iniciar sesión; (2) escribir un SSID propio, identificable y autorizado; (3) elegir un canal; (4) pulsar *Desplegar SoftAP*; (5) verificar en la OLED la IP, el SSID y el número de clientes; y (6) probar el portal desde un teléfono de prueba. Si el sistema operativo no abre la ventana automática, se puede navegar a `http://192.168.4.1`. El portal no debe usarse para imitar redes de terceros ni para capturar datos sin consentimiento.

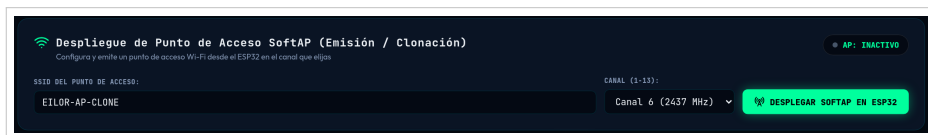


Figura 16.4: Configuración del SoftAP: SSID, canal y activación controlada desde el dashboard.

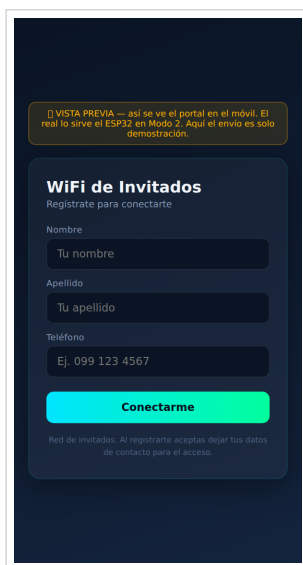


Figura 16.5: Portal cautivo que visualiza el invitado al conectarse a la red propia.

16.1.8. Registros de invitados

La sección *Registros del Portal Cautivo* lista las altas (nombre, apellido, teléfono, red y fecha) en tiempo real. Desde allí se puede *Exportar CSV* o *Borrar* los registros (ambas acciones requieren sesión).

Antes de exportar, el operador debe verificar que la captura fue informada y consentida. El archivo CSV debe almacenarse con permisos restringidos y eliminarse cuando finalice el periodo de retención definido por la organización. La acción *Borrar* es irreversible desde la interfaz, por lo que se recomienda exportar solo cuando exista una finalidad legítima y una copia controlada.

FECHA / HORA	NOMBRE	APELLIDO	TELÉFONO	RED (SSID)
9/23/2026, 6:25:10 AM	Jorge	Vélez	096 555 1212	Cafe-EILOR
9/23/2026, 6:25:10 AM	Lucía	Andrade	097 222 3344	Cafe-EILOR
9/23/2026, 6:25:10 AM	Carlos	Mena	098 765 4321	Cafe-EILOR
9/23/2026, 6:25:10 AM	Ana	Torres	099 111 2233	Cafe-EILOR

Figura 16.6: Registros del portal cautivo y controles de exportación y limpieza.

16.1.9. Indicadores y telemetría

La tarjeta de telemetría confirma la última muestra, el estado del enlace y la actividad del dispositivo. Una demora aislada no implica una falla: primero se debe observar si el contador se actualiza después del siguiente heartbeat. La vista de dispositivos ayuda a distinguir redes detectadas, balizas BLE y registros del portal; no debe interpretarse como una captura del contenido de las comunicaciones.

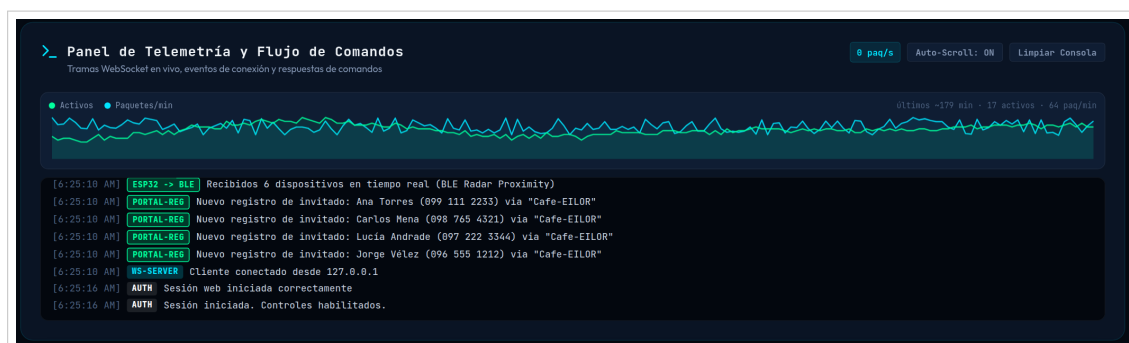


Figura 16.7: Panel de telemetría y estado de la conexión del dispositivo.



Figura 16.8: Vista completa del *dashboard* con todas sus secciones (cabecera, controlador de modos, SoftAP, espectro, radar, telemetría, inventario y registros).

16.2. Manual técnico

16.2.1. Flujo técnico de extremo a extremo

El firmware inicia Wi-Fi en modo estación, configura I²C en GPIO21/GPIO22, inicializa la OLED en la dirección 0x3C y registra el pulsador en GPIO15 con INPUT_PULLUP. Después establece el WebSocket TLS hacia el servidor. Cada ciclo de escaneo produce un mensaje JSON con tipo de medición, modo, identificador de dispositivo y resultados; el servidor valida, persiste y difunde la actualización a los navegadores conectados.

16.2.2. Estructura del proyecto

```

1 eilor/
2   server.js           # servidor Express + WebSocket
3   lib/               # modulos: analysis, oui, store, auth
4   public/            # dashboard: index.html, app.js, styles.css,
                        portal-preview.html
5   firmware/          # eilor_esp32.ino (firmware del ESP32)
6   data/              # persistencia JSON (se crea sola; en .
                        gitignore)
7   package.json

```

Listing 16.1: Estructura de directorios

16.2.3. Instalación y ejecución del servidor

```

1 cd eilor
2 npm install           # instala express, ws, cors
3 EILOR_PASSWORD="tu_clave" node server.js
4 # o gestionado con PM2:
5 pm2 start server.js --name eilor && pm2 save

```

Listing 16.2: Puesta en marcha del servidor

16.2.4. Variables de entorno

Tabla 16.1: Variables de entorno del servidor

Variable	Función	Valor por defecto
PORT	Puerto de escucha	3002
HOST	Interfaz de escucha	127.0.0.1
EILOR_PASSWORD	Contraseña del dashboard	eilor2024
EILOR_DEVICE_TOKEN	Token para restringir la ingesta	(sin definir: ingesta abierta)

16.2.5. Carga del firmware

El firmware se compila y sube desde Arduino IDE o PlatformIO. Debe ubicarse en una carpeta con su mismo nombre, junto al archivo `images.h`. Antes de compilar se configuran las credenciales Wi-Fi, el `WS_SERVER_HOST` y, opcionalmente, el `DEVICE_TOKEN` y el nombre del portal (`PORTAL_TITULO`). Librerías requeridas: `WebSockets` (Markus Sattler), `Adafruit_SSD1306`, `Adafruit_GFX` y `ezButton`.

Después de cargar el firmware, abrir el monitor serie a 115200 baudios y comprobar, en orden, la inicialización de la OLED, la asociación Wi-Fi, la conexión TLS y los mensajes de cambio de modo. Si la pantalla no inicia, comprobar VCC, GND, SDA,

SCL y la dirección I²C antes de volver a grabar. Para actualizar varias unidades se debe conservar el mismo identificador lógico del dispositivo únicamente cuando cada unidad tenga su propio token y registro de inventario.

16.2.6. Prueba técnica de aceptación

La siguiente lista permite cerrar una instalación con evidencia reproducible:

1. El servidor inicia sin errores y responde en el puerto configurado.
2. El dashboard permite iniciar sesión y cambia a solo lectura cuando se cierra sesión.
3. El ESP32 aparece en línea y envía al menos un heartbeat.
4. El modo Wi-Fi actualiza redes y el radar BLE actualiza dispositivos.
5. El botón cambia entre los cuatro modos sin rebotes visibles.
6. El SoftAP entrega una IP, muestra clientes en la OLED y presenta el portal.
7. Un registro sin conexión queda en cola y se reenvía al recuperar el WebSocket.
8. La exportación CSV y la limpieza solo funcionan con sesión autenticada.

La evidencia mínima debe incluir una captura de la cabecera, una del modo de escaneo, una del portal y una del registro recibido. Estas imágenes se conservan junto con la fecha, versión del firmware y configuración del servidor.

16.2.7. Publicación con Cloudflare Tunnel

```
1 cloudflared tunnel create eilor-tunnel
2 cloudflared tunnel route dns eilor-tunnel esp.tu-dominio.com
3 cloudflared tunnel run eilor-tunnel    # usa el config.yml del Cap. 8
```

Listing 16.3: Publicación del servicio con cloudflared

16.2.8. Resolución de problemas frecuentes

- **OLED apagada o con basura:** comprobar 3V3/GND, invertir SDA/SCL solo si el módulo lo exige y verificar la dirección 0x3C.
- **ESP32 reinicia al escanear:** sustituir el cable o fuente por una capaz de entregar al menos 500 mA y comprobar que no se estén usando dos fuentes.
- **ESP32 marca "sin conexión":** verificar credenciales Wi-Fi, hora del sistema, alcance del túnel y que el servidor esté en línea (`pm2 logs eilor`).

- **El botón cambia dos modos:** revisar el cable entre GPIO15 y GND y mantener el antirrebote de 50 ms del firmware.
- **El portal no aparece:** confirmar que el dispositivo está en modo 2 y abrir `http://192.168.4.1`.
- **Registros no llegan:** revisar el enlace WebSocket y el contador de clientes; el *buffer* reenviará al reconectar mientras no se alcance su capacidad.
- **Ingesta rechazada (401):** el servidor tiene EILOR_DEVICE_TOKEN definido; configurar el mismo token en el firmware.
- **Dashboard sin datos:** recargar el navegador, confirmar la sesión y revisar que el puerto del servidor coincida con el configurado en el túnel.

16.2.9. Mantenimiento, respaldo y seguridad

Antes de modificar firmware o servidor se debe copiar la carpeta `data/`, el archivo de configuración del túnel y la versión actualmente operativa. Las credenciales Wi-Fi, la contraseña del dashboard y el token del dispositivo no deben publicarse en el repositorio. Se recomienda rotarlos cuando cambie el operador, limitar el acceso al dashboard mediante HTTPS y revisar periódicamente los registros almacenados. La carcasa debe abrirse con la alimentación desconectada y las pilas deben retirarse antes de intervenir el cableado.

Capítulo 17

Conclusiones y recomendaciones

Este capítulo presenta el cierre general del proyecto EILOR. Las conclusiones integran los resultados funcionales, técnicos y formativos, y explicitan las limitaciones que deben considerarse al interpretar el prototipo. Finalmente se formulan recomendaciones concretas para su uso responsable y evolución.

17.1. Conclusiones generales y limitaciones

1. **La solución cumple el objetivo general planteado.** EILOR integra un ESP32, un servidor Node.js y un dashboard web para observar la banda de 2.4 GHz, identificar dispositivos y administrar una red propia de invitados. Las pruebas funcionales confirmaron el escaneo Wi-Fi, el radar BLE, el SoftAP, el portal cautivo y la actualización en tiempo real. La conclusión se limita a las tecnologías y condiciones evaluadas; el prototipo no pretende sustituir un analizador de espectro profesional ni una auditoría especializada.
2. **La arquitectura de tres capas es viable, reproducible y de bajo costo.** La comunicación mediante WebSocket, persistencia local y publicación mediante túnel TLS permitió obtener una mediana de 2.5 ms entre la ingesta y la difusión de eventos, con un costo estimado de hardware de USD 24.20 por unidad. La persistencia en archivos JSON resulta adecuada para una demostración y una instalación pequeña, pero limita las consultas, la concurrencia y la retención cuando aumenta el volumen de datos.
3. **Los indicadores de radio son útiles para diagnóstico relativo.** El puntaje de interferencia, la recomendación de canales, el RSSI, el catálogo OUI y la detección de SSID repetidos aportan información operativa para localizar congestión y cambios en el entorno. Sin embargo, el RSSI no proporciona una distancia métrica precisa y

la heurística de gemelo malicioso puede producir falsos positivos en redes mesh o falsos negativos ante una clonación más avanzada.

4. **La seguridad implementada protege el flujo básico, pero no alcanza un perfil de producción endurecido.** La validación de sesión y token de dispositivo, las respuestas 401/200 y el transporte TLS/WSS reducen el acceso no autorizado en el escenario probado. Persisten limitaciones: la contraseña del dashboard requiere un mecanismo de hashing y control de intentos, los tokens deben rotarse y el despliegue depende de la protección de las credenciales y de la configuración del túnel.
5. **El principal valor de EILOR es formativo y de gestión responsable.** El proyecto hace observables conceptos como interferencia, BLE, OUI, aleatorización de MAC, portal cautivo y autenticación, con hardware accesible y código auditable. Su uso válido se restringe a redes propias, información de baliza públicamente radiada y registros obtenidos con aviso y consentimiento; no cubre 5 GHz/6 GHz, no decodifica tráfico de terceros y no autoriza la suplantación de redes ajenas.

17.2. Recomendaciones

1. **Endurecer la autenticación antes de cualquier despliegue real:** almacenar contraseñas con hashing y sal, limitar intentos de inicio de sesión, rotar periódicamente el token del dispositivo y mantener un registro de auditoría.
2. **Migrar la persistencia cuando crezca el volumen:** sustituir los archivos JSON por SQLite en instalaciones pequeñas o PostgreSQL en la nube, conservando la interfaz de `lib/store` para no acoplar la lógica de negocio al motor de almacenamiento.
3. **Calibrar y documentar cada instalación:** registrar ubicación, orientación de la antena, tipo de carcasa, fuente de alimentación y condiciones del entorno; interpretar RSSI y recomendación de canal como tendencias relativas, no como mediciones absolutas.
4. **Aplicar un protocolo de privacidad y operación:** usar únicamente SSID propios e identificables, mostrar el aviso del portal, solicitar los datos mínimos, restringir las exportaciones CSV, definir un periodo de retención y retirar las credenciales del código fuente antes de publicar el firmware.
5. **Ampliar la validación de forma gradual:** probar el sistema durante periodos prolongados y con varios dispositivos, comparar resultados en distintos entornos y, solo después, evaluar soporte multi-dispositivo, analítica histórica, cobertura de 5 GHz y escalamiento del WebSocket mediante un broker.

En conjunto, EILOR queda validado como prototipo funcional, económico y didáctico para monitoreo responsable de redes propias. Las recomendaciones definen las condiciones mínimas para pasar de una demostración controlada a una operación más robusta, segura y mantenible.

Referencias bibliográficas

- Akyildiz, I. F., Lee, W.-Y., Vuran, M. C., & Mohanty, S. (2006). NeXt generation/dynamic spectrum access/cognitive radio wireless networks: A survey. *Computer Networks*, 50(13), 2127-2159. <https://doi.org/10.1016/j.comnet.2006.05.001>
- Bahl, P., & Padmanabhan, V. N. (2000). RADAR: An in-building RF-based user location and tracking system. *Proceedings IEEE INFOCOM 2000*, 2, 775-784. <https://doi.org/10.1109/INFCOM.2000.832252>
- Bianchi, G. (2000). Performance analysis of the IEEE 802.11 distributed coordination function. *IEEE Journal on Selected Areas in Communications*, 18(3), 535-547. <https://doi.org/10.1109/49.840210>
- Bluetooth SIG. (2023). *Bluetooth Core Specification, Version 5.4* (inf. téc.). Bluetooth Special Interest Group. Kirkland, WA.
- Callegati, F., Cerroni, W., & Ramilli, M. (2009). Man-in-the-middle attack to the HTTPS protocol. *IEEE Security & Privacy*, 7(1), 78-81. <https://doi.org/10.1109/MSP.2009.12>
- Espressif Systems. (2023). *ESP32 Series Datasheet, Version 4.3*. Espressif Systems. Shanghai, China.
- Espressif Systems. (2026). *ESP32-DevKitC V4 User Guide*. Consultado el 23 de septiembre de 2026, desde https://docs.espressif.com/projects/esp-dev-kits/en/latest/esp32/esp32-devkitc/user_guide.html
- Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol* (RFC N.º 6455). Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC6455>
- Fielding, R. T., & Taylor, R. N. (2002). Principled design of the modern Web architecture. *ACM Transactions on Internet Technology*, 2(2), 115-150. <https://doi.org/10.1145/514183.514185>
- Gast, M. S. (2005). *802.11 Wireless Networks: The Definitive Guide* (2.^a ed.). O'Reilly Media.
- Heydon, R. (2013). *Bluetooth Low Energy: The Developer's Handbook*. Prentice Hall.
- IEEE. (2021). *IEEE Standard for Information Technology — Telecommunications and Information Exchange between Systems — Local and Metropolitan Area Networks*

- *Specific Requirements — Part 11: Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) Specifications* (IEEE Std N.º 802.11-2020). Institute of Electrical y Electronics Engineers. <https://doi.org/10.1109/IEEESTD.2021.9363693>
- IEEE Registration Authority. (2017). *Guidelines for use of Organizationally Unique Identifier (OUI) and Company ID (CID)*. Consultado el 23 de septiembre de 2026, desde <https://standards.ieee.org/products-programs/regauth/>
- Kurose, J. F., & Ross, K. W. (2017). *Computer Networking: A Top-Down Approach* (7.^a ed.). Pearson.
- Martin, J., Mayberry, T., Donahue, C., Foppe, L., Brown, L., Riggins, C., Rye, E. C., & Brown, D. (2017). A study of MAC address randomization in mobile devices and when it fails. *Proceedings on Privacy Enhancing Technologies*, 2017(4), 365-383. <https://doi.org/10.1515/popets-2017-0054>
- Mockapetris, P. (1987). *Domain Names — Implementation and Specification* (RFC N.º 1035). Internet Engineering Task Force (IETF). <https://doi.org/10.17487/RFC1035>
- OpenJS Foundation. (2024). *Node.js Documentation*. Consultado el 23 de septiembre de 2026, desde <https://nodejs.org/en/docs>
- OWASP Foundation. (2021). *OWASP Top Ten 2021*. Consultado el 23 de septiembre de 2026, desde <https://owasp.org/Top10/>
- Rappaport, T. S. (2002). *Wireless Communications: Principles and Practice* (2.^a ed.). Prentice Hall.
- Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5.^a ed.). Pearson.
- Yücek, T., & Arslan, H. (2009). A survey of spectrum sensing algorithms for cognitive radio applications. *IEEE Communications Surveys & Tutorials*, 11(1), 116-130. <https://doi.org/10.1109/SURV.2009.090109>

Apéndice A

Glosario de términos

BLE	Bluetooth Low Energy; variante de bajo consumo del estándar Bluetooth.
BSSID	Identificador único (dirección MAC) de un punto de acceso Wi-Fi.
Evil twin	Punto de acceso que suplanta el SSID de una red legítima.
Heatmap	Mapa de calor; representación de la ocupación por canal.
OUI	Organizationally Unique Identifier; prefijo de fabricante en la MAC.
Portal cautivo	Página que intercepta la navegación de un cliente recién conectado.
RSSI	Received Signal Strength Indicator; indicador de intensidad de señal.
SoftAP	Punto de acceso Wi-Fi emitido por software desde el propio dispositivo.
SSID	Nombre público de una red Wi-Fi.
WebSocket	Protocolo de comunicación bidireccional persistente sobre TCP.
WSS	WebSocket sobre TLS (cifrado).

Apéndice B

Referencias del código fuente

El código fuente completo del sistema acompaña a este documento. La Tabla B.1 enumera los archivos principales y su extensión.

Tabla B.1: Archivos de código fuente del proyecto

Archivo	Líneas	Descripción
firmware/eilor_esp32.ino	777	Firmware del ESP32 (4 modos, portal cautivo, buffer)
server.js	665	Servidor Express + WebSocket + API REST
lib/analysis.js	71	Puntaje de canal y detección de gemelo malicioso
lib/oui.js	62	Fabricante por OUI y MAC aleatorias
lib/store.js	81	Persistencia JSON y series temporales
lib/auth.js	60	Sesiones por token y autorización
public/app.js	913	Lógica del dashboard en tiempo real
public/index.html	419	Estructura del dashboard
public/styles.css	1272	Estilos del tablero

Apéndice C

Repositorio y despliegue de referencia

El código fuente completo del proyecto (firmware, servidor, módulos y frontend) se aloja en el siguiente repositorio, que acompaña a este documento como anexo digital:

- **Repositorio de código:** <https://github.com/ptandazoz9827/evil-esp32>
- **Plataforma en producción:** <https://esp.orellanadigitalco.trade/>
- **Túnel y hostname público:** `esp.orellanadigitalco.trade` (puerto 443, WSS/HTTPS), publicado con Cloudflare Tunnel (Capítulo 10).

La plataforma en producción integra el flujo de extremo a extremo descrito en el Capítulo 11: el dispositivo ESP32 se conecta por WSS al servidor a través del túnel, y los clientes web consumen el *dashboard* sobre HTTPS. Para reproducir el despliegue basta clonar el repositorio, instalar las dependencias (`npm install`), definir `EILOR_PASSWORD` y ejecutar el servidor bajo PM2, siguiendo el manual técnico del Capítulo 16.